République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École Nationale d'Électronique
et des Télécommunications de Sfax

Ingénieur en :
Génie des Télécommunications

Projet De Fin D'Année

N° d'ordre : PFA-GT-2026-29

Projet De Fin D'Année

Présenté à

L'École Nationale d'Électronique et des Télécommunications de Sfax

Filière :

Génie des Télécommunications

par

Seifeddine EL ABED
Aymen GHARBI

Sécurisation D'une Infrastructure Zero-Trust

Soutenu le : 22/04/2026, devant la commission d'examen :

M. Nouri OMHENI	Rapporteur
M. Faouzi ZARAI	Encadrant

DEDICATION

À nos familles,
Pour leur amour inconditionnel, leur soutien indéfectible et leurs encouragements constants qui nous ont guidés à chaque étape de notre parcours.

À nos enseignants et encadrants,
Pour leurs conseils précieux, leur patience et leur engagement à nous transmettre leur savoir.

À nous-mêmes,
Pour avoir osé rêver, persévéré face aux obstacles et s'être engagés avec passion dans cette aventure. Pour notre comité, le soutien mutuel et l'inspiration que nous nous sommes apportés, faisant de ce chemin un moment de vie inoubliable.

À nos amis,
Pour leur présence, leur bienveillance et les moments de partage qui ont rendu cette aventure encore plus enrichissante.

À tous ceux qui, de près ou de loin,
ont contribué à la réalisation de ce travail, nous nous adressons notre profonde reconnaissance.

À **nous tous**,
Nous dédions ce travail avec toute notre gratitude.

Seifeddine El Abed & Aymen Gharbi

REMERCIEMENTS

Avant de mettre en valeur notre projet, nous souhaitons réserver ces quelques lignes pour exprimer notre profonde gratitude et remercier toutes les personnes qui ont contribué à la réalisation de ce travail.

Tout d'abord, nous tenons à exprimer nos sincères remerciements à notre encadrant académique, M. Faouzi ZARAI, pour son suivi attentif, sa disponibilité et ses précieux conseils tout au long de ce projet. Son engagement et ses orientations pertinentes nous ont été d'une grande aide dans l'aboutissement de notre travail.

Nous sommes également honorés par l'intérêt porté à notre projet par les membres du jury, qui ont accepté de l'évaluer et de juger notre travail.

Nous souhaitons également exprimer notre profonde gratitude envers l'École Nationale d'Électronique et des Télécommunications de Sfax (ENET'COM), notre établissement de formation, qui nous a offert un cadre propice à l'apprentissage et au développement de nos compétences. Nous remercions particulièrement l'ensemble du corps enseignant pour la qualité de l'enseignement dispensé, leur engagement et leur soutien tout au long de notre parcours académique.

Enfin, nous adressons nos plus sincères remerciements à nos familles et amis, pour leur soutien moral, leurs encouragements constants et leur patience durant cette aventure.

Seifeddine El Abed & Aymen Gharbi

TABLE DES MATIÈRES

TABLE DES MATIÈRES ii

LISTE DES FIGURES vii

LISTE DES TABLEAUX viii

LISTE DES ABRÉVIATIONS ix

INTRODUCTION GENERALE 1

I État de l'Art 3

I.1 Introduction . . . 4

I.2 Fondements théoriques . . . 4

I.2.1 Historique synthétique et évolution vers le cloud . . . 4

I.2.2 Modélisation de la surface d'attaque . . . 4

I.2.3 Modèle zero Trust . . . 5

I.2.4 Technologies de défense cloud . . . 5

I.2.4.1 Listes de Contrôle d'Accès (*NSG*) . . . 5

I.2.4.2 Pare-Feux Périmétrique (*Firewall*) . . . 8

I.2.4.3 *Web Application Firewall* (*WAF*) . . . 10

I.2.4.4 Réseaux Privés Virtuels (*VPN*) . . . 13

I.3 Exemples d'attaques . . . 15

I.4 Approches *Zero Trust* existantes et travaux antérieurs . . . 16

I.4.1 Travaux antérieurs sur la sécurisation via *IaC* . . . 16

I.4.2 Approches *Zero Trust* existantes . . . 16

I.5 Justification des choix technologiques . . . 17

I.5.1 Introduction Globale des arbitrages . . . 17

I.5.2 Choix de l'isolation segmentée (NSG) . . . 17

I.5.3 Choix du pare-feu Egress (Azure Firewall) . . . 17

I.5.4 Choix de la prévention applicative (WAF v2) . . . 17

I.5.5 Choix de l'accès distant (Passerelle Route-based VPN Point-to-site) . . . 18

I.6 Synthèse et valeurs ajoutées attendues 18
I.7 Conclusion 20

II Mise en place d'une solution de sécurisation 21
II.1 Introduction 22
II.2 Conception et architecture 22
II.2.1 Processus du projet 22
II.2.2 Description de l'infrastructure 23
II.3 Installation et configuration 24
II.3.1 Environnement de mise en place 24
II.3.2 Installation 25
II.3.3 Configuration 26
II.3.3.1 Préparation et validation Terraform 26
II.3.3.2 Provisionnement des ressources 27
II.3.3.3 Création de pipeline de déploiement avec GitHub Actions . . 27
II.3.3.4 Configuration de la VM avec Docker pour le déploiement . . 28
II.3.3.5 Configuration DNS et HTTPS 28
II.3.3.6 Configuration VPN 28
II.4 Resultats 29
II.5 Test avec les attaques 31
II.5.1 Attaques manuelles 31
II.5.2 Attaques automatisées 33
II.5.2.1 Attaque automatisée 1: Balayage initial (Nikto) 34
II.5.2.2 Attaque automatisée 2: SQL Injection (SQLMap) 35
II.5.2.3 Attaque automatisée 3: Fuzzing (Wfuzz) 36
II.5.3 Analyse synthétique des résultats 37
II.6 Problèmes résolus & remédiations 37
II.7 Conclusion 40

CONCLUSION GENERALE 41

BIBLIOGRAPHIE 43

ANNEXES 45
A.1 Integration et Configuration du VPN Point-to-Site 45
A.1.1 Etape 1 : Generation du package client 45
A.1.2 Etape 2 : Installation du client VPN 46

A.1.3 Etape 3 : Preparation pour l'import du profil 46
A.1.4 Etape 4 : Selection du fichier de configuration 47
A.1.5 Etape 5 : Import des parametres de connexion 47
A.1.6 Etape 6 : Validation du profil importe 48
A.1.7 Etape 7 : Etablissement de la connexion VPN 48
A.1.8 Etape 8 : Verification finale de la connexion 49
A.1.9 Configuration et verification des ressources VPN 49
A.2 Tracabilite des actifs images 51
A.3 Processus d'achat et configuration DNS / TLS via Namecheap 51
A.3.1 Configuration initiale et preuves d'integration 51

LISTE DES FIGURES

I.1 Architecture d'un NSG (Listes de Contrôle d'Accès) 8
I.2 Fonctionnement d'un Pare-Feu Périmétrique 10
I.3 Fonctionnement d'un Web Application Firewall (WAF) 12
I.4 Architecture d'un Réseau Privé Virtuel (VPN) 15

II.1 Structure globale justifiant le changement architectural 23
II.2 Architecture cible sécurisée adoptée intégrant tous les contrôles 24
II.3 Initialisation et authentification Azure CLI 26
II.4 Préparation et validation Terraform 26
II.5 Provisionnement des ressources via Terraform 27
II.6 Création de pipeline de déploiement avec GitHub Actions (YAML) 27
II.7 Configuration - Bloc Terraform client P2S 28
II.8 Resultat - Fin de déploiement Terraform réussi 29
II.9 Resultat - Preuve de déploiement dans Azure Portal 29
II.10 Resultat - Execution validee du workflow GitHub Actions 30
II.11 Resultat - Enregistrements de validation de domaine 30
II.12 Resultat - Preuve de connexion SSH 1 via le tunnel VPN 31
II.13 SQLi - Soumission du payload d'attaque 32
II.14 SQLi - Journaux WAF témoignant du blocage (Matched Rule) 32
II.15 XSS - Soumission du payload d'attaque 33
II.16 XSS - Journaux WAF n'interceptant pas le vecteur d'attaque 33
II.17 Automatisation - Exécution du balayage Nikto 34
II.18 Automatisation - Journaux interceptant le balayage Nikto 34
II.19 Automatisation - Exécution de SQLMap 35
II.20 Automatisation - Journaux confirmant le blocage massif de SQLMap 35
II.21 Automatisation - Exécution de Wfuzz 36
II.22 Automatisation - Journaux validant l'interception Wfuzz 36
II.23 Diagramme - Trajectoire de réalisation de la remédiation 39
II.24 Diagramme - Maturité défensive de l'architecture finale 39

A.1 VPN client - Etape 1: generation du package 45
A.2 VPN client - Etape 2: installation Azure VPN Client 46
A.3 VPN client - Etape 3: interface et preparation import 46
A.4 VPN client - Etape 4: selection azurevpnconfig.xml 47
A.5 VPN client - Etape 5: parametres importes 47
A.6 VPN client - Etape 6: profil importe 48
A.7 VPN client - Etape 7: connexion etablie 48
A.8 VPN client - Etape 8: verification finale 49
A.9 VPN - Gateway deploye dans Azure 49
A.10 VPN - Parametres Point-to-Site OpenVPN 50
A.11 VPN - Regle de securite NSG bloquante par defaut 50
A.12 VPN - Verification des regles NSG dans le portail Azure 50
A.13 VPN - Preuve SSH 1 via tunnel 51
A.14 Application - Acces IP publique avant domaine (Alerte TLS) 52
A.15 TLS - Certificat valide sur l'entree publique 52
A.16 Domaine 1 - Securisation HTTPS validee 53
A.17 Domaine 2 - Securisation HTTPS validee 53
A.18 Mobile - Accessibilite securisee sur le domaine 54

LISTE DES TABLEAUX

I.1 Positionnement théorique de la segmentation NSG 6
I.2 Positionnement théorique d'un Firewall Cloud 8
I.3 Positionnement théorique du WAF 11
I.4 Positionnement théorique du VPN Infonuagique 13
I.5 Problèmes cibles et valeur ajoutée technique attendue 19

II.1 Caractéristiques du matériel utilisé 24
II.2 Pile logicielle et outils utilisés 25

LISTE DES ABRÉVIATIONS

IaaS Infrastructure as a Service

PaaS Platform as a Service

SaaS Software as a Service

IaC Infrastructure as Code

WAF Web Application Firewall

NSG Network Security Groups

VNet Virtual Network

OWASP Open Web Application Security Project

XSS Cross-Site Scripting

SQLi SQL Injection

DDoS Distributed Denial of Service

SIEM Security Information and Event Management

CI/CD Continuous Integration / Continuous Deployment

DevOps Development Operations

JWT JSON Web Token

PKI Public Key Infrastructure

TLS Transport Layer Security

HTTPS HyperText Transfer Protocol Secure

SSH Secure Shell

RBAC Role-Based Access Control

CIS Center for Internet Security

NIST National Institute of Standards and Technology

GDPR General Data Protection Regulation

PFA Projet de Fin d'Année

RFC Request for Comments

URL Uniform Resource Locator

DNS Domain Name System

FQDN Fully Qualified Domain Name

VM Virtual Machine

RG Resource Group

SKU Stock Keeping Unit

API Application Programming Interface

REST Representational State Transfer

JSON JavaScript Object Notation

HCL HashiCorp Configuration Language

CLI Command Line Interface

INTRODUCTION GENERALE

La cybersécurité est aujourd'hui un enjeu transversal qui dépasse largement le cadre d'un secteur ou d'une technologie précise, car la numérisation des services, l'usage massif des API, la migration continue vers le cloud, la multiplication des terminaux connectés et l'automatisation des chaînes de livraison ont transformé les systèmes d'information en environnements fortement distribués, dynamiques et donc plus difficiles à maîtriser de bout en bout.

Dans ce contexte général, les incidents se répètent selon des motifs bien connus du domaine : erreurs de configuration exposées sur Internet, segmentation insuffisante, privilèges excessifs, secrets mal gouvernés, supervision partielle, réponse tardive aux alertes, traitement réactif plutôt que préventif, et absence de boucle formelle entre détection, correction et validation. Ces problèmes répétitifs ne relèvent pas uniquement d'un manque d'outils ; ils traduisent surtout un écart récurrent entre la complexité des architectures modernes et la maturité des pratiques de sécurité appliquées au quotidien.

La tendance de fond est donc claire : la cybersécurité moderne exige une approche systémique, mesurable et continue, où la protection n'est pas un composant ajouté à la fin, mais une qualité intégrée aux choix de conception, aux mécanismes de déploiement et aux routines d'exploitation. C'est à partir de cette lecture générale du domaine que se pose la problématique qui guide ce mémoire : comment construire une infrastructure cloud qui reste non seulement fonctionnelle et performante, mais aussi défendable dans la durée, c'est-à-dire capable de réduire sa surface d'attaque, de contenir les effets d'une tentative d'exploitation, de produire des preuves exploitables, puis d'améliorer sa posture par une remédiation contrôlée ?

Autrement dit, la question centrale n'est pas de savoir si une attaque est possible, mais comment concevoir un système qui anticipe ce risque, encaisse les écarts et transforme chaque test en apprentissage opérationnel.

C'est dans ce cadre que s'inscrit notre projet de fin d'année, qui consiste à concevoir et implémenter une architecture cloud sécurisée. Dans cette perspective, le projet s'appuie sur une architecture Azure segmentée [1], un plan de sécurisation multicouche (WAF, NSG, firewall stateful, VPN d'administration, gouvernance TLS/Key Vault), une orchestration Terraform [2] pour garantir la cohérence des changements, ainsi qu'une validation offensive par des campagnes manuelles et automatisées afin de mesurer l'effet réel des choix retenus.

Ce positionnement permet d'adresser les compromis classiques du domaine, notamment l'équilibre entre protection et disponibilité, entre granularité de contrôle et maintenabilité, entre vitesse de déploiement et rigueur de validation, ainsi qu'entre détection d'anomalies et capacité de remédiation sans régression.

Sur le plan de la démarche, ce mémoire adopte volontairement une posture d'ingénierie : observer l'existant, identifier les besoins de sécurité prioritaires, proposer une solution argumentée, adopter une architecture cible soutenable, puis prouver les gains à travers des résultats techniques traçables. Cette orientation répond directement aux attentes d'un travail de fin d'année en télécommunications et cybersécurité, où la qualité du raisonnement, la robustesse des décisions et la lisibilité des preuves sont aussi importantes que le résultat final.

Pour rendre cette progression claire, le rapport est organisé en deux chapitres principaux après cette introduction générale :

Le chapitre I, consacré à l'état de l'art et aux fondements théoriques, présente les notions structurantes du domaine (historique des approches de défense, modèles de risque, principes d'architecture sécurisée, modèles Zero Trust), illustre des exemples d'attaques, puis formalise l'analyse de l'existant et des besoins afin de justifier de façon détaillée les choix technologiques retenus.

Le chapitre II détaille la mise en place d'une solution de sécurisation, depuis la matérialisation de l'infrastructure et des contrôles de sécurité jusqu'aux preuves de validation, aux attaques exécutées, aux logs observés et aux corrections appliquées, afin de mettre en évidence la valeur concrète de la démarche adoptée.

Enfin, ce rapport se clôture par une conclusion générale qui synthétise les acquis, explicite les limites du projet et ouvre des perspectives d'amélioration vers une maturité sécuritaire encore plus élevée.

chapitre

I État de l'Art

PLAN DU CHAPITRE

I.1. Introduction

I.2. Fondements théoriques

I.3. Exemples d'attaques

I.4. Approches Zero Trust existantes et travaux antérieurs

I.5. Justification des choix technologiques

I.6. Synthèse et valeurs ajoutées attendues

I.7. Conclusion

I.1 Introduction

Ce chapitre établit les bases scientifiques et méthodologiques de notre projet. Il débute par la présentation des fondements théoriques liés à la sécurité *Cloud* et au modèle *Zero Trust* [1]. Ensuite, nous exposons les principales menaces à travers des exemples d'attaques concrets, avant de passer en revue les approches existantes et les travaux antérieurs sur la sécurisation des infrastructures via l'*IaC* (*Infrastructure as Code*) [2]. Forts de cet état de l'art, nous présentons la solution adoptée. Enfin, nous justifions les choix technologiques retenus et présentons une synthèse des valeurs ajoutées attendues.

I.2 Fondements théoriques

La cybersécurité *Cloud* s'est construite par paliers successifs : périmètre réseau classique, sécurité des applications web, virtualisation, puis modèles *Zero Trust* et *Security as Code* [1]. Cette section détaille de manière approfondie l'évolution des pratiques ainsi que les modèles conceptuels et mathématiques qui justifient les mesures de sécurité contemporaines en milieu *Cloud*.

I.2.1 Historique synthétique et évolution vers le cloud

Les premières approches se concentraient sur le filtrage entrant simple et les antivirus périmétriques, traitant le réseau comme un espace de confiance dont seules les bordures devaient être protégées. Avec l'avènement des architectures web et la prolifération des vulnérabilités applicatives, de nouveaux outils ont été imposés tels que les *Web Application Firewalls* (*WAF*), la micro-segmentation et la journalisation centralisée [3]. Aujourd'hui, la tendance dominante s'appuie sur la défense en profondeur, l'isolation par défaut, la gouvernance granulaire des identités, la rotation rigoureuse des secrets, et surtout, l'automatisation en "*Infrastructure as Code*" (*IaC*) pour réduire de manière significative les erreurs humaines et les dérives de configuration [2].

I.2.2 Modélisation de la surface d'attaque

La surface d'attaque globale peut être évaluée mathématiquement par la somme pondérée de toutes les ouvertures publiques ou internes :

$$S = \sum_{i=1}^{m} w_i \cdot e_i \tag{I.1}$$

Dans ce cas, la variable e_i décrit un point d'exposition spécifique qu'un attaquant peut solliciter (un port de management laissé ouvert, une API non authentifiée, ou encore une identité mal gérée). La pondération w_i correspond à la sévérité ou au niveau de privilège atteignable en exploitant l'entrée e_i. L'objectif fondamental d'une architecture infonuagique sécurisée - et le but de notre projet - est donc de minimiser S de façon chirurgicale en neutralisant les e_i inutiles, sans dégrader la disponibilité fonctionnelle de l'application [3].

I.2.3 Modèle zero Trust

Le principe historique du "périmètre de confiance" est désormais obsolète. Le principe du moindre privilège impose que chaque identité, chaque processus de service et chaque flux de données réseau ne dispose que des permissions strictement minimales nécessaires à son opération. Le modèle "Zero Trust" radicalise cette logique en considérant qu'aucun domaine du réseau n'est par défaut fiable, ordonnant qu'absolument toute requête soit vérifiée en continu indépendamment de sa provenance géographique ou de sa position dans le sous-réseau "interne" [1]. Il repose sur trois piliers fondamentaux :

- Vérification explicite et continue de l'identité, du contexte réseau et de la conformité du terminal émetteur.
- Accès restreint limité au double principe du "*Just-In-Time*" (droits temporaires) et "*Just-Enough-Access*" (droits limités).
- Assomption systématique de la compromission ("*Assume Breach*"), exigeant une forte segmentation des *workloads Cloud* pour empêcher qu'une machine corrompue n'accède aux données des autres machines adjacentes.

I.2.4 Technologies de défense cloud

Pour concrétiser l'approche de défense en profondeur au sein d'une architecture *Cloud*, il est indispensable de déployer un ensemble de solutions de sécurité complémentaires. Ces composants fondamentaux, détaillés ci-dessous, assurent conjointement le filtrage réseau bas niveau, l'inspection applicative approfondie et l'accès chiffré.

I.2.4.1 Listes de Contrôle d'Accès (*NSG*)

Les Groupes de Sécurité Réseau (*NSG*) constituent une première ligne de défense vitale pour la ségrégation granulaire au sein d'une infrastructure infonuagique. Ce sont des filtres de paquets statiques virtuels qui traitent le trafic entrant et sortant avant d'atteindre le système d'exploitation de la cible. En examinant systématiquement les flux réseaux sur la base de contrôles stricts (*IP* et ports), ils interdisent ou autorisent explicitement les données sans

nécessiter d'appliance physique. Les Groupes de Sécurité Réseau (*NSG*) constituent une première ligne de défense vitale pour la ségrégation granulaire au sein d'une infrastructure infonuagique. Ce sont des filtres de paquets statiques virtuels qui traitent le trafic entrant et sortant avant d'atteindre le système d'exploitation de la cible. En examinant systématiquement les flux réseaux sur la base de contrôles stricts (*IP* et ports), ils interdisent ou autorisent explicitement les données sans nécessiter d'appliance physique [1].

Le Tableau I.1 présente cette couche de contrôle, limitée à une inspection de surface centrée sur les adresses *IP* et les ports, tout en jouant un rôle essentiel dans la segmentation initiale du trafic.

Classification	**Type d'inspection**
Profondeur d'inspection	Couches réseau et transport (*IP* source/destination, ports)
Méthode d'inspection	Règles statiques priorisées (Autoriser/Refuser)
Menaces ciblées	Exposition accidentelle de ports, mouvements asymétriques inter-subnets

Table I.1: *Positionnement théorique de la segmentation NSG*

- **Types :**
  - **NSG au niveau du sous-réseau (*Subnet-level*) :** Se rattachent globalement à un *sous-réseau* entier, garantissant qu'aucune entité ne puisse outrepasser la règle établie pour le groupe d'adresses, ce qui facilite la gestion de masse et la gouvernance.
  - **NSG au niveau de l'interface réseau (*NIC-level*) :** S'attachent de façon très granulaire à une carte réseau virtuelle spécifique, permettant des politiques sur mesure pour une machine unique qui se trouverait au sein d'un plus vaste sous-réseau où des règles génériques ne suffisent pas.
  - **Groupes de sécurité d'application (*Application Security Groups - ASG*) :** Permettent de regrouper des interfaces réseau logiquement selon leur fonction applicative plutôt que sur des plages d'adresses *IP* rudimentaires complexes.
- **Avantages / Inconvénients :**

  Pour les NSG au niveau du sous-réseau (*Subnet-level*) :
  - *Avantage :* Facilitent une politique de gouvernance globale en assurant la protection unifiée d'une multitude de serveurs simultanément, et d'un calcul de flux virtuellement gratuit par l'infrastructure matérielle sous-jacente.

  - *Inconvénient :* L'application aveugle des restrictions interdit de pouvoir adapter exceptionnellement des ports isolés pour un seul serveur précis au profil applicatif divergeant dans cet espace.

Pour les NSG au niveau de l'interface réseau (NIC-level) :

  - *Avantage :* Répondent favorablement au manque de singularité en permettant d'associer une restriction sur mesure stricte à l'entrée d'une interface spécifique, ignorant les limites de la segmentation physique en place.
  - *Inconvénient :* S'avèrent difficiles à maintenir à grande échelle: la multiplication des configurations individuelles augmente la complexité opérationnelle et le risque d'erreur.

Pour les Groupes de sécurité d'application (ASG) :

  - *Avantage :* Accroissent remarquablement la clarté opérationnelle lors des déploiements majeurs centralisés, permettant d'affecter simplement l'étiquette "Machine Web" afin de leur propager implicitement l'ouverture globale sur le protocole de couche transport (L4) exigé.
  - *Inconvénient :* La cécité contextuelle "stateless" (couche réseau/L3 et couche transport/L4) perdure toujours : ils ignoreront à vie de statuer sur le corps HTTPS du port dont l'ouverture applicative aura été certifiée, permettant l'ingérence en continu d'attaquants via un port légitime.

- **Niveau de filtrage + type de trafic :** Les *NSG* concentrent leur puissance de traitement préférentiellement sur les niveaux *ISO* L3 (Couche Réseau) et L4 (Couche Transport). Ils sont primordiaux pour verrouiller techniquement les communications internes Est-Ouest entre différents sous-réseaux adjacents du *Cloud*, et encadrent grossièrement le filtrage basique Nord-Sud pour les requêtes publiques via les ports standards, tels que l'acceptation rudimentaire du port 443 *TCP*.

- **Limites :** L'impossibilité inhérente d'inspecter les requêtes jusqu'au palier *HTTP/HTTPS* (données applicatives (couche 7)) reste majeure : une machine affectée par un *malware* et utilisant un port autorisé pourra transmettre des ordres *C2*. De plus, ils ne corrélent pas d'états asymétriques, laissant les entités d'un même sous-réseau exposées aux mouvements latéraux si l'une d'entre elles vient à être compromise depuis une surface adjacente.

La Figure I.1 montre l'illustration correspondante.

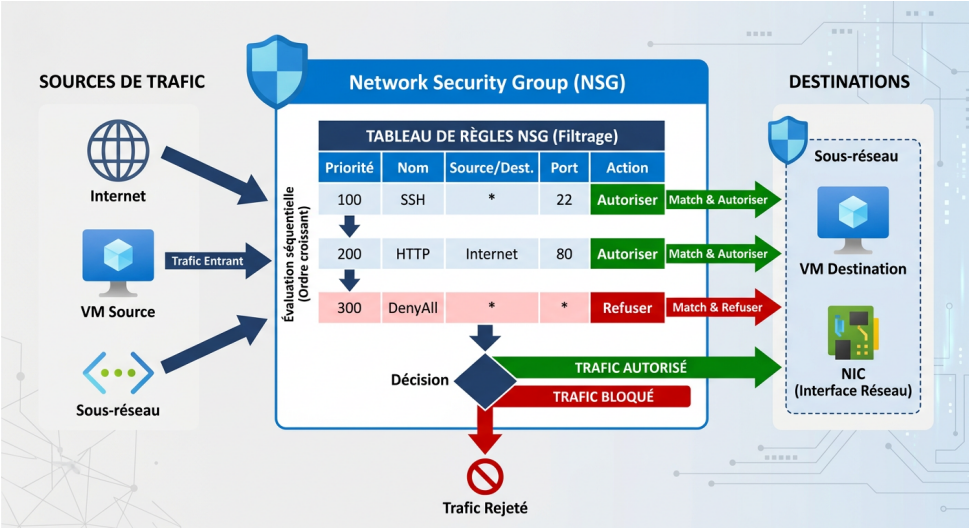


Figure I.1: *Architecture d'un NSG (Listes de Contrôle d'Accès)*

I.2.4.2 Pare-Feux Périmétrique (*Firewall*)

Le Pare-feu Périmétrique Infonuagique se pose comme une instance réseau logicielle centralisée agissant tel un goulot d'étranglement unifié pour inspecter les flux routés. Différent du *NSG*, il interprète l'état algorithmique complet de l'échange (*Stateful*) pour bloquer les incursions opportunistes sur des connexions établies artificiellement. Il garantit ainsi un contrôle accru sur les communications sortantes et inter-réseaux. Le Pare-feu Périmétrique Infonuagique se pose comme une instance réseau logicielle centralisée agissant tel un goulot d'étranglement unifié pour inspecter les flux routés. Différent du *NSG*, il interprète l'état algorithmique complet de l'échange (*Stateful*) pour bloquer les incursions opportunistes sur des connexions établies artificiellement. Il garantit ainsi un contrôle accru sur les communications sortantes et inter-réseaux [1].

Les détails figurent dans le Tableau I.2, le pare-feu périmétrique élargit l'inspection au maintien d'état et aux flux applicatifs de base, ce qui le distingue clairement d'un simple filtrage réseau.

Classification	**Type d'inspection**
Profondeur d'inspection	Couches réseau et transport + données d'application basiques
Méthode d'inspection	Table d'états (*Stateful*), Règles réseaux et applicatives
Menaces ciblées	Exfiltration furtive de données privées, mouvements latéraux contournés

Table I.2: *Positionnement théorique d'un Firewall Cloud*

- **Types :**

- **Pare-feu à états (*Stateful Firewall*) :** Maintient en mémoire constante une table d'états qui surveille la création (*SYN*, *ACK*) et la fermeture des sessions réseaux, identifiant toute trame n'appartenant pas légitimement à une communication approuvée préalablement afin de réduire les manœuvres asynchrones non sollicitées.
- **Pare-feu de Nouvelle Génération (*NGFW*) :** Va considérablement plus loin en amalgamant des fonctions de pare-feu avancées associées à des techniques d'inspection approfondie des paquets réseau (*DPI*) ou des Systèmes de Prévention d'Intrusion (*IPS/IDS*). Cela lui permet ainsi de scanner activement des logiciels malveillants au cœur même du trafic traversé par des protocoles complexes.
- **Pare-feu de *Proxy* d'application (*Proxy Firewall*) :** S'interpose littéralement comme relai exclusif appliquant une coupure drastique ("*Air-Gap*" logiciel) pour isoler la cible en recréant systématiquement sa propre communication.

- **Avantages / Inconvénients :**

  Pour le Pare-feu à état (*Stateful Firewall*) :

  - *Avantage :* Apporte une surveillance de contexte asymétrique qui bloque l'effraction de manière économique, couplée à sa fiabilité permettant la centralisation claire du routage *Egress* avec *proxy DNS* sans latence de calcul.
  - *Inconvénient :* Bloqué à la vision architecturée réseau *IP*, il passe inéluctablement à côté d'anomalies de la couche application: la légitimité *IP* d'un flux n'évitera aucune attaque qui cible et corrompt la composante *HTTP* de l'hôte.

  Pour le Pare-feu de Nouvelle Génération (*NGFW*) :

  - *Avantage :* Fournit une souveraineté massive grâce à la gouvernance active. Son inspection *DPI* offre le super-pouvoir de désamorcer préventivement non seulement les *malwares*, mais d'interdire des requêtes *Web* d'une menace inconnue.
  - *Inconvénient :* Implique la location d'une ressource extrêmement onéreuse (*CAPEX/OPEX* démesurés comparés à l'*Instance* applicative protégée), qui provoque potentiellement un goulot de ralentissement majeur "*Choke-point*" lors du décapsulage crypto systématique du trafic de couche application (L7) intense.

  Pour le Pare-feu de *Proxy* d'application (*Proxy Firewall*) :

  - *Avantage :* L'ingénierie qui brise complètement la chaîne de requêtes empêche catégoriquement les attaques de protocole *IP* (ex: *ICMP* malformés, requêtes *DoS* ciblées du tunnel).
  - *Inconvénient :* Interfère fortement avec presque tout trafic en streaming réel, souffre de contraintes applicatives sur la réécriture de certificats *Web* et impose de constants paramétrages spécifiques contraignants.

- **Niveau de filtrage + type de trafic :** Le dispositif accomplit des merveilles quant au filtrage intelligent de la couche transport (L4) vis-à-vis du trafic d'administration et interne, tout en ayant de fortes capacités limitées mais utiles de bloquage sur la couche application (L7) via les mots clefs *FQDN*. Cela lui permet avant tout de policer et de rejeter drastiquement tout type de sorties non sollicitées vers des *URL* néfastes pour garantir l'assainissement global *Egress*.
- **Limites :** Malgré ses filtres contextuels, ce dernier n'embarque pas d'analyse des signatures relatives au trafic applicatif hébergé. Il échouera catégoriquement face aux injections encodées ou lors des requêtes *HTTP* illégitimes visant à compromettre un formulaire *TLS* chiffré, requérant inévitablement un *WAF* pour la complétude de protection.

Voir la Figure I.2 pour l'illustration correspondante.

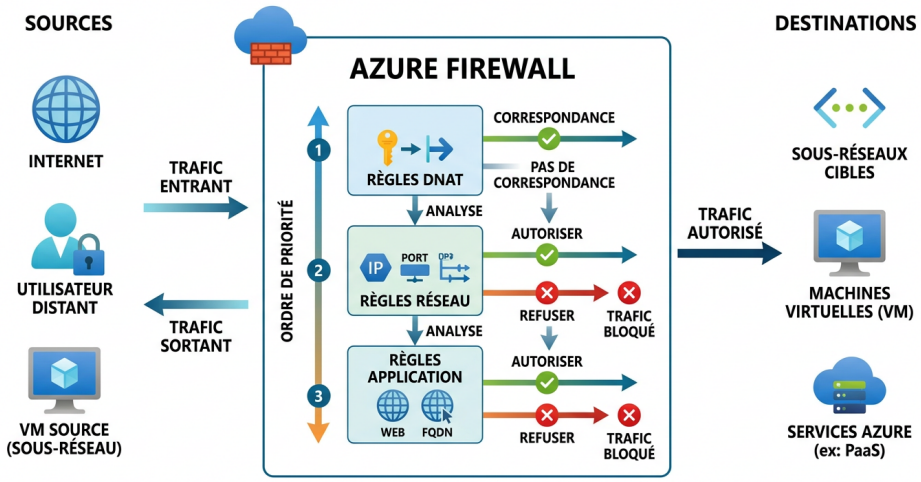


Figure I.2: *Fonctionnement d'un Pare-Feu Périmétrique*

I.2.4.3 *Web Application Firewall* (*WAF*)

Contrairement aux pare-feux conventionnels, le *Web Application Firewall* (*WAF*) s'érige comme un bouclier spécialisé conçu exclusivement pour fonctionner en *Reverse-Proxy* sur le trafic *Web*. Il assume la responsabilité de déchiffrer les requêtes *TLS*, d'analyser finement les en-têtes *HTTP*, les *URI* et les *payloads* pour neutraliser les attaques applicatives (*SQLi*, *XSS*) [3]. C'est seulement suite à une validation sémantique méticuleuse qu'il transmettra l'approbation au serveur visé.

Voir le Tableau I.3 pour le rôle du WAF, qui intervient au niveau applicatif profond afin de détecter et bloquer les attaques Web ciblées avant qu'elles n'atteignent le serveur.

Classification	Type d'inspection
Profondeur d'inspection	Inspection approfondie des données d'application (*header proxy*, *URI*, *param*, corps)
Méthode d'inspection	Base de signatures heuristiques (*OWASP CRS*), détection d'anomalies
Menaces ciblées	Défiguration logicielle, *OWASP Top* 10 [3] (*SQLi*, *CSRF*, *RCE*)

Table I.3: *Positionnement théorique du WAF*

- **Types :**
  - ***WAF* basé *Appliance* (*VM* tierce) :** Nécessite le déploiement manuel d'une machine virtuelle provenant d'un éditeur de sécurité tiers, s'exécutant indépendamment des conteneurs *Web* visés.
  - ***WAFaaS* natif *Cloud* (*as-a-Service*) :** Est directement mutualisé de façon logiciellement gérée (*Managed Service*) par le *Cloud Provider* au niveau de la passerelle de livraison, interceptant le public.
  - ***WAF* basé sur l'hôte (*Host-based*) :** S'installe comme un agent léger ou un module rattaché directement en profondeur (*plug-in*) au sein du service de fonctionnement *Web* local (le code source d'*Apache*, *Nginx*).
- **Avantages / Inconvénients :**

  Pour le *WAF* basé *Appliance* (*VM* tierce) :
  - *Avantage :* Alloue une isolation complète du trafic via un système d'exploitation sécurisé hermétique autorisant une gouvernance sur-mesure d'ingénierie très pointue, ainsi que des modifications du noyau logiciel indépendant.
  - *Inconvénient :* Occasionne des temps gigantesques de paramétrages lors de chaque architecture, exige une mise en œuvre complexe du *Load-Balancing* en haute-disponibilité ("*Scaling*" coûteux et très lent).

  Pour le *WAFaaS* natif *Cloud* (*as-a-Service*) :
  - *Avantage :* L'activation fournit rapidement une protection mutualisée ("*Core Rule Sets*") contre le "*Top* 10 *OWASP*", grâce à sa position en amont et à une distribution géographique adaptée.
  - *Inconvénient :* Implique un contrôle algorithmique obscur (dépendance d'éditeur *SaaS* en "Boîte Noire") dont la sensibilité mal configurée (faux-positifs) écarte

arbitrairement des visiteurs adoptant par accident un trafic qualifié comme "suspect" à ses heuristiques, et intercepte la clef de chiffrement privée sur un réseau qui n'est plus chez le client.

Pour le *WAF* basé sur l'hôte (*Host-based*) :

- *Avantage :* Favorise radicalement la proximité défensive: il peut consulter chaque action locale post-déchiffrement *SSL* et de le lier intimement à la pile en garantissant virtuellement la nullité en latence de routage externe.
- *Inconvénient :* Dévore d'énormes parts de performances sur le serveur cible. En cas de surcharges (*DDoS* ou pics normaux), c'est l'hôte lui-même qui souffre et dont la chute du service devient inévitable sans protection externe préalable ("*Denial-of-Service*").

- **Niveau de filtrage + type de trafic :** Le *WAF* dédie exclusivement le 100% imposant de sa puissance calculatoire pour l'inspection hyper-profondie rattachée à la couche application (L7) applicative. Il concentre son champ de vigilance de manière intraitable sur tous les appels dits Nord-Sud, et se dévoue à inspecter rigoureusement les verbes sensibles *POST/GET* provenant des flux publics *Ingress* demandant accès au réseau de la charge de travail logicielle.

- **Limites :** Le *WAF* est conçu pour les requêtes *Web*. Il ne couvre pas les agressions menées sur du *TCP* brut, telles qu'une altération de port de base de données interne, et doit donc être complété par des *NSG* et des contrôles réseau en aval.

On retrouve ce résultat à la Figure I.3.

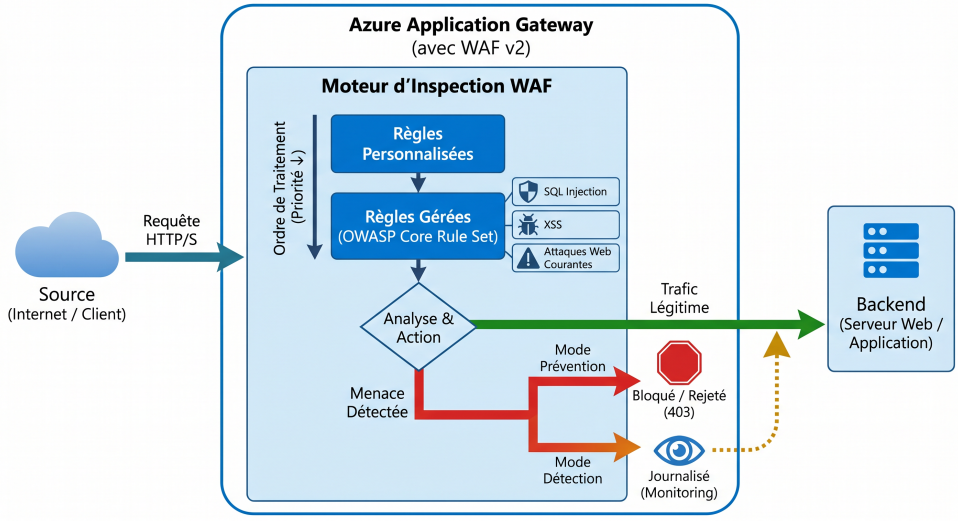


Figure I.3: *Fonctionnement d'un Web Application Firewall (WAF)*

I.2.4.4 Réseaux Privés Virtuels (*VPN*)

Conçu pour neutraliser la menace d'un accès d'administration externe vulnérable, le Réseau Privé Virtuel (*VPN*) installe une technologie purement cryptographique en réseau superposé. Par le biais du chiffrement lourd du canal d'accès transitant sur *Internet*, il autorise un utilisateur nomade authentifié à rejoindre le réseau privé en sécurité. Cela permet de masquer totalement les interfaces de maintenance (*RDP*, *SSH*) face aux scanners publics [1].

On retrouve ce point dans le Tableau I.4, le VPN se positionne comme une couche d'accès chiffrée destinée à protéger les connexions d'administration et à masquer les services sensibles.

Classification	**Type d'inspection**
Profondeur d'inspection	Couches réseau et transport
Méthode d'inspection	Encapsulation de paquets chiffrés (*IPSec/OpenVPN*)
Menaces ciblées	Usurpation d'identifiant, interception de trafic d'administration

Table I.4: *Positionnement théorique du VPN Infonuagique*

- **Types :**
  - ***VPN* de Site à Site (*Site-to-Site VPN*) :** Tunnel permanent établi en continu entre une passerelle physique de l'entreprise locale et la ressource infonuagique cible, fusionnant le routage de chaque équipement de l'*intranet*.
  - ***VPN* de Point à Site (*Point-to-Site VPN*) :** Définie une connexion d'accès asynchrone qui s'effectue sporadiquement à la volonté d'un client logiciel déployé sur une seule machine autorisée (idéal au profil nomade).
  - ***VPN* infonuagique (*Cloud-to-Cloud / VNet-to-VNet*) :** Sorte d'association transparente au sein de l'ossature *IP* ("*Backbone*" *Cloud*) facilitant le transfert crypté de données intercontinentales rattachées au fournisseur.
- **Avantages / Inconvénients :**

  Pour le *VPN* de Site à Site (*Site-to-Site VPN*) :
  - *Avantage :* Fournit l'intégration massive de l'activité du pôle bureau ("*Headquarters*") au pôle Serveur web: une passerelle suffit à la transmission sans authentification singulière redondante d'un réseau tout entier, à haut-débit.
  - *Inconvénient :* Implique la gestion vitale de configurations matérielles (Routeur *IPSec*, *BGP*) et nécessite des contrats fournisseurs *Internet* colossaux. Son arrêt brise en un point critique toute liaison du bloc entreprise, paralysant logistiquement les équipes de techniciens.

Pour le *VPN* de Point à Site (*Point-to-Site VPN*) :

- *Avantage :* Alloue une souveraineté irréprochable au télétravail ("*SSTP/OpenVPN*"). Le périmètre d'infogérance masque radicalement ses portails vulnérables (22 *SSH*, 3389 *RDP*) et contredit activement le danger originel de la *Brute-force* d'un compte ouvert au net, remplaçant la confiance en adresse *IP* par un profil *Microsoft Entra* de "certificat public/privé".
- *Inconvénient :* Il ne cloisonne pas la charge interne : ce type de passe d'accès n'exigeant pas formellement les règles de *Zero Trust* de couche application (L7). La perméabilité d'une compromission (*Ransomware* local du client nomade) se déplacera vers l'infrastructure privée, en "tunnelant" son activité malveillante au travers de l'enceinte de protection originelle sans heurts.

Pour le *VPN* infonuagique (*Cloud-to-Cloud / VNet-to-VNet*) :

- *Avantage :* Garanti une latence d'action pratiquement nulle entre des instances situées mondialement, supprimant drastiquement les embargos cryptographiques tiers et la charge fastidieuse d'une instance virtuelle de routage *BGP*.
- *Inconvénient :* Accroît le piège opérationnel irrémédiable des "adresses réseau concomitantes": connecter un réseau A (10.0.0.0/8) à un réseau B (10.0.0.0/8) est impossible et exige un travail acharné d'architecture dès le premier bloc afin que chaque masque sous-réseau respecte infailliblement une segmentation indépendante et globale sans chevauchement.

- **Niveau de filtrage + type de trafic :** Visant préférentiellement et globalement une politique fonctionnant sur encapsulation protocolaire (*tunneling* couche réseau/L3), le dispositif traite purement le trafic d'infogérance et intervient pour canaliser les demandes confidentielles des techniciens *IT* afin de permettre leurs connexions privilégiées invisibles qui entrent au niveau dorsal du domaine.

- **Limites :** Dans de trop grands nombres de schémas imparfaits, la présence même du *VPN* garantit un accès complet, sans pour autant le borner. La faille n'étant plus sur *Internet*, l'infiltration naîtrait en profitant de l'absence de vérification en aval au sein de l'*intranet*, exposant alors brutalement tous les serveurs à une compromission pivot si le *Zero Trust* ne bloque pas le *VPN* par des filtres de privilèges croisés.

La Figure I.4 illustre ce point.

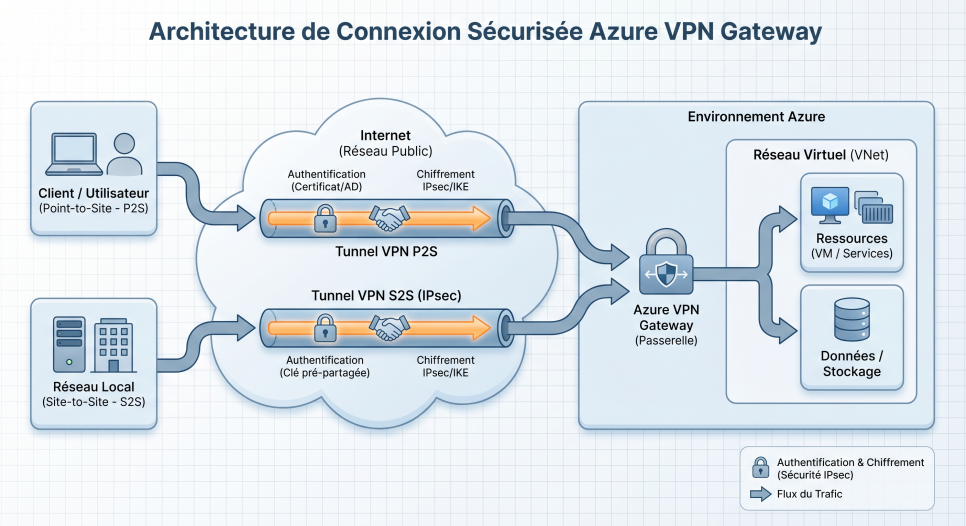


Figure I.4: *Architecture d'un Réseau Privé Virtuel (VPN)*

I.3 Exemples d'attaques

La littérature sécuritaire foisonne de cas d'exploitations infonuagiques qui profitent des failles listées dans les sections abordées juste au-dessus. Pour concevoir judicieusement nos contrôles cloud, il convient de s'appuyer sur la définition précise des scénarios d'incursions destructeurs majeurs rencontrés en entreprises :

- **Injections SQL (SQLi) et Cross-Site Scripting (XSS) :** L'injection classique constitue une exploitation flagrante de mauvaise hygiène du code. Lorsqu'un endpoint Web échoue à nettoyer une saisie (input validation absente ou paramètre REST), un attaquant falsifie malicieusement les données saisies afin qu'elles soient injectées et compilées par le serveur interne. Lors d'un SQLi, cela amène à la fuite massive de la base de données privée de l'entreprise en extrayant tables et mots de passe hashés. Le XSS en revanche injecte du Javascript dans des sites réputés légitimes et s'exécute dans les navigateurs des collaborateurs pour subtiliser les cookies à distance. Ces exécutions furtives contournent aisément les réseaux fermés, illustrant l'absolue nécessité d'activer un dispositif défensif de couche application (L7 - WAF) pour bloquer ces requêtes à priori légitimes [3].

- **Mouvements latéraux (Lateral Movement) :** Lorsqu'un attaquant compromet une passerelle (ex. le serveur DMZ) via n'importe laquelle des méthodes Web précitées, la seconde phase opératoire est le Pivot, soit l'expansion horizontale vers les sous-réseaux voisins, à l'aveugle ou à l'aide de scanners (comme Nmap). L'absence de barrières (NSG) en intra-réseau crée le phénomène d'une intrusion singulière qui engendre la compromission globale de toute la plateforme Cloud, les ressources fonctionnant dans un modèle archaïque de confiance implicite du voisinage IP [4].

- **Exfiltration de données (Data Exfiltration) :** Une attaque furtive ne laisse pas d'anomalies de rupture réseau derrière elle : un malware installé initialement ordonnera silencieusement à la machine Cloud de transférer périodiquement (vers un serveur Pirate Control-and-Command - C2) d'énormes fichiers chiffrés contenant les bases de l'entreprise ciblée. Souvent dissimulée dans les ports non régulés du trafic sortant (Egress HTTPS), elle contourne toutes les barrières entrantes.

- **Attaques réseaux et protocolaires sur les ports d'administration :** Sans canal virtuel privé (VPN) pour cacher les moyens de contrôle, l'exposition publique des ports de maintenance (le port SSH 22, RDP 3389) au domaine Internet provoque un balayage automatisé des botnets par "Brute-Force". Ces tentatives brutales engorgeront inévitablement les serveurs jusqu'à ce que des milliers de combinaisons brisent la logique d'authentification des techniciens systèmes faibles.

I.4 Approches *Zero Trust* existantes et travaux antérieurs

Avant d'aborder en détail les travaux antérieurs et les approches existantes, il convient de souligner l'importance de ces concepts dans le contexte actuel de la sécurité *Cloud*. Cette partie met en évidence les stratégies adoptées pour faire face aux défis de sécurité émergents.

I.4.1 Travaux antérieurs sur la sécurisation via *IaC*

L'*Infrastructure as Code* (*IaC*) a transformé le déploiement *Cloud*. Les travaux antérieurs montrent que l'intégration de la sécurité directement dans les fichiers *Terraform* (tests statiques, *linting* de sécurité, gestion des états décentralisés) prévient la majorité des vulnérabilités avant déploiement [2]. L'*IaC* n'est plus seulement un outil d'automatisation, c'est un mécanisme de conformité (*Policy-as-Code*).

I.4.2 Approches *Zero Trust* existantes

Les environnements *Cloud* modernes adoptent le *Zero Trust* à différents niveaux [1] :

- ***Zero Trust Network Access* (*ZTNA*) :** Remplace les *VPN* traditionnels en fournissant un accès granulaire aux applications sans exposer l'ensemble du réseau (*Software-Defined-Perimeter*).

- **Micro-segmentation *Cloud-native* :** Utilisation d'identités logiques (*Service Principals*, *Managed Identities*) au lieu de simples identités adresses *IP* pour autoriser formellement les communications inter-services explicites.

- **Gouvernance continue :** Évaluation dynamique et automatisée des risques (via *Azure Defender*) qui déclenche une levée d'accès ou des remédiations lorsque l'environnement est compromis en temps réel.

I.5 Justification des choix technologiques

I.5.1 Introduction Globale des arbitrages

Après l'analyse des besoins de défense, la validation des paramètres de base et l'esquissage théorique de notre solution adoptée, la sélection des outils pour l'implémenter a été orientée pour offrir le meilleur triptyque pragmatique de la cybersécurité moderne : robustesse du filtrage, maintenabilité réseau et orchestration as Code.

I.5.2 Choix de l'isolation segmentée (NSG)

Le choix des Network Security Group (NSG) appliqués en intra-zones justifie une politique fondamentale : la "séparation inter-fonctionnelle". Renoncer à déployer un matériel Appliance lourd, pour privilégier l'option native NSG, permet l'avantage inestimable d'écrire la restriction logique absolue au plus proche des instances de calcul sans latence ou sur-coût financier. Ceci crée le bouclier parfait contre la prolifération silencieuse des dommages internes.

I.5.3 Choix du pare-feu Egress (Azure Firewall)

Sélectionner spécifiquement Azure Firewall Standard et non un routeur open-source standard s'avère un parti-pris capital : sa structure offre une Haute Disponibilité as-a-Service et un ajustement d'échelle natif transparent (auto-scaling) face aux attaques. Par ailleurs, la puissance conceptuelle de bloquer logiciellement un trafic malicieux furtif sortant est permise nativement par ses capacités d'application Threat Intelligence, rendant notre infrastructure conforme aux normes de prévention sur les Exfiltrations de serveurs internes sans recourir à un proxy Egress onéreux supplémentaire.

I.5.4 Choix de la prévention applicative (WAF v2)

Azure Application Gateway avec le WAF V2 a été privilégié en point d'entrée Internet pour concentrer les vulnérabilités de la couche 7. Plutôt que de bâtir des règles Web applicatives incertaines au sein même de nos serveurs, l'App Gateway WAFv2 incorpore structurellement le déchiffrement certifié asymétrique SSL puis l'inspection syntaxique formelle des requêtes sous le bouclier de sûreté OWASP CRS. Le choix de WAF est justifié car sa position intercepte les

injections SQL/XSS avant le serveur de donnée, et limite le risque induit par le Single Point of Failure applicatif.

I.5.5 Choix de l'accès distant (Passerelle Route-based VPN Point-to-site)

L'incorporation de bout-en-bout du VPN fonctionnant avec les accès vérifiés "Microsoft Entra ID" dans sa méthode "Point-to-Site VPN", fut sélectionnée rigoureusement au détriment d'une "Jump Box" (VM Bastion publique Linux classique). Un accès VPN IKEv2 offre l'atout conceptuel de rayer d'Internet les ports à hauts risques de "télégestion" des parcs cloud. Un hacker ne peut plus scanner les entrées d'infogérance car celles-ci requièrent une cryptographie asymétrique "Client-to-Core-Cloud".

I.6 Synthèse et valeurs ajoutées attendues

L'intérêt technique fondamental du parcours d'ingénierie que nous venons d'explorer se mesure de manière synthétique sur son alignement tactique vis-à-vis des failles primitives visées tout au long des précédentes sections. Le Tableau I.5 met en évidence la valeur défensive concrète et mesurable apportée à chaque pallier du modèle Zero Trust.

Problème de sécurité structurelle cible	**Choix décisionnel de conception théorique**	**Avantage défensif et Valeur Ajoutée**
Dérive de configuration de production manuelle ("*Configuration Drift*")	Automatisation *Terraform* via *Infrastructure As Code*	Immutabilité continue, Reproductibilité exacte du code versionné à chaque déploiement.
Sur-exposition du profil du réseau intra-*cloud* par défaut	Micro-segmentation par *subnet* et interface avec règles *NSG* limitées	Réduction drastique des vulnérabilités par mouvements asymétriques latéraux, limitant le rayon d'impact interne.
Failles applicatives et abus protocolaires *Web HTTP/HTTPS*	*WAF Cloud-based* (*OWASP Core* 3.2, règles de signatures) en mode prévention	Imposition sécuritaire en amont pour bloquer les attaques pathogènes courantes (injection *SQL*, *XSS*, etc.).
Sorties suspectes ou exfiltration par une *VM* interne (vers *C2*)	Goulot contrôlé (*Choke-Point Firewall Standard stateful*) via *UDR egress*	Assure un verrouillage qualitatif de chaque requête émanant du système, bloquant la fuite du trafic *web DNS* pirate.
Sécurisation béante et obsolète des terminaux d'administration à distance	Canal *Cloud* privé virtuel *VPN Routing* (*Point-To-Site*) avec authentification *Entra ID*	Fermeture immédiate des terminaux d'exploitation *SSH* sur la face *internet*. Renforce la confidentialité des accès d'administration à distance.

Table I.5: *Problèmes cibles et valeur ajoutée technique attendue*

- **Interprétation :** La stratégie *Zero Trust* déployée repose sur une défense en profondeur superposant *firewall* applicatif (*WAF*), micro-segmentation (*NSG*) et contrôle des flux sortants (*Azure Firewall*). L'accès d'administration *VPN* chiffré et l'orchestration *as Code* (*IaC*) consolident cet écosystème en réduisant la surface d'attaque et en garantissant une résilience face aux menaces modernes.

I.7 Conclusion

Ce chapitre a établi un état de l'art du modèle *Zero Trust* et de l'*IaC* face aux menaces *cloud* modernes. L'analyse des concepts a permis de justifier l'architecture défensive retenue (*Azure*, *Terraform*, *WAF*, pare-feux, *VPN*). Le chapitre suivant détaillera l'implémentation technique et la validation de cette infrastructure par des scénarios d'attaque offensifs.

chapitre

II Mise en place d'une solution de sécurisation

PLAN DU CHAPITRE

II.1. Introduction

II.2. Conception et architecture

II.3. Installation et configuration

II.4. Resultats

II.5. Test avec les attaques

II.6. Problèmes résolus & remédiations

II.7. Conclusion

II.1 Introduction

Nous avons utilisé l'outil de balayage de vulnérabilités Nikto [5]. Dès le déclenchement de la commande, le trafic massif et non standard généré par Nikto a sollicité les mécanismes de sécurité de bordure. Ce chapitre présente de manière structurée la mise en œuvre technique du projet, depuis le déploiement de l'infrastructure jusqu'à la validation offensive et la remédiation des écarts. Les éléments présentés visent à établir une traçabilité claire entre configuration appliquée, preuve de fonctionnement et valeur sécuritaire obtenue.

II.2 Conception et architecture

II.2.1 Processus du projet

L'intervention d'ingénierie de ce projet se calque le long d'un processus strict :

1. **Analyse des Actifs et Conceptions :** Identification précise des ressources informatiques pour concevoir un socle théorique limitant les dégâts de compromissions.
2. **Conception Déclarative Modulaire (IaC) :** Coder des modules déclaratifs rigoureux isolant logiciellement nos besoins de déploiement (WAF, Firewall, VPN, Subnets).
3. **Implémentation Cloud :** Transposition dans le Cloud avec l'API Azure pour valider techniquement la stabilité de l'environnement infonuagique provisionné.
4. **Campagnes d'Attaques de test et Remédiations :** Validation pragmatique en lançant virtuellement de vrais scripts d'agression sur le périmètre WAF ou VPN mis en place afin de mesurer ou de raffiner nos règles du Zero Trust à sa juste valeur.

Dans un objectif de restitution claire, l'architecture infra-cloud finale de cette conception théorique est validée par la modélisation affichée sur la Figure II.1.

La Figure II.1 montre l'illustration correspondante.

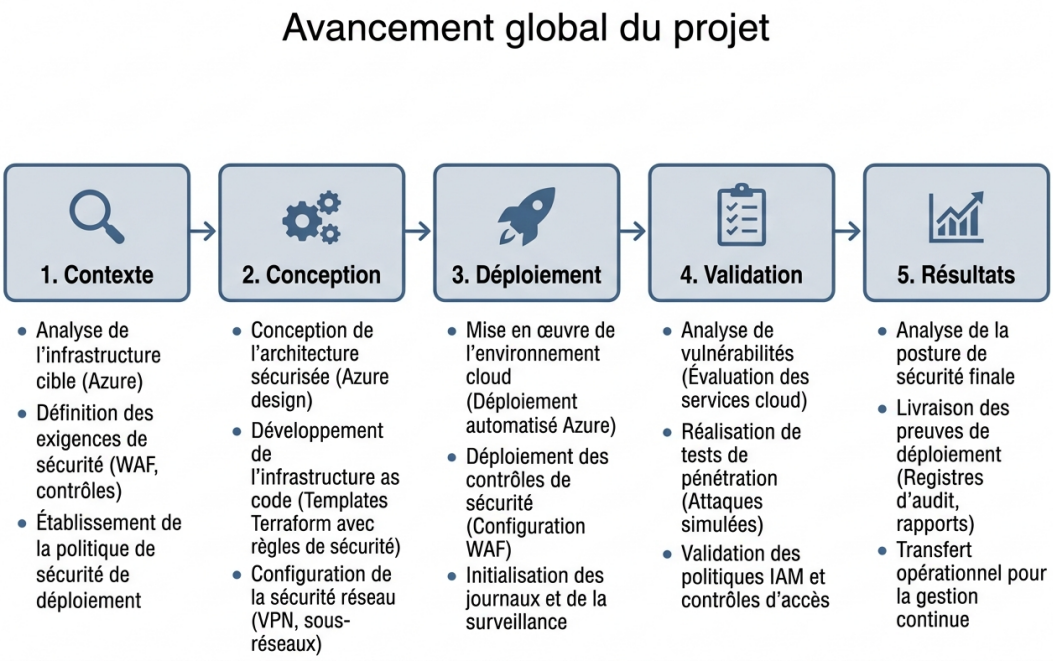


Figure II.1: *Structure globale justifiant le changement architectural*

II.2.2 Description de l'infrastructure

En analysant la topologie de la solution adoptée (Figure II.2: Architecture cible sécurisée adoptée intégrant tous les contrôles), l'infrastructure se stratifie en sous-réseaux (Subnets) dédiés (Application Gateway, Subnet VM Runner, Subnet de base de données Back-End). Le trafic public externe de requêtes HTTP/S ne rejoint, par logique de routage, que le Front-End Application Gateway équipé du WAF, configuré pour rejeter les menaces OWASP avant tout accès au back-end. De la même façon, tout processus interne des machines voulant engendrer des connexions de sortie web (exfiltration malveillante ou mise à jour non prévue) est intercepté et contraint via la table de route UDR vers Azure Firewall Standard. Au sein du nuage en interne (trafic dit Est-Ouest), l'intégralité des serveurs déployés interconnectés est isolée via des listes NSG, refusant par défaut les tentatives de mouvement latéral non autorisées. En définitive, une passerelle Virtual Network Gateway annexe (Point-To-Site VPN) accorde un tunneling chiffré permettant au responsable Cloud de télé-administrer le réseau dans un cadre maîtrisé.

Voir la Figure II.2 pour l'illustration correspondante.

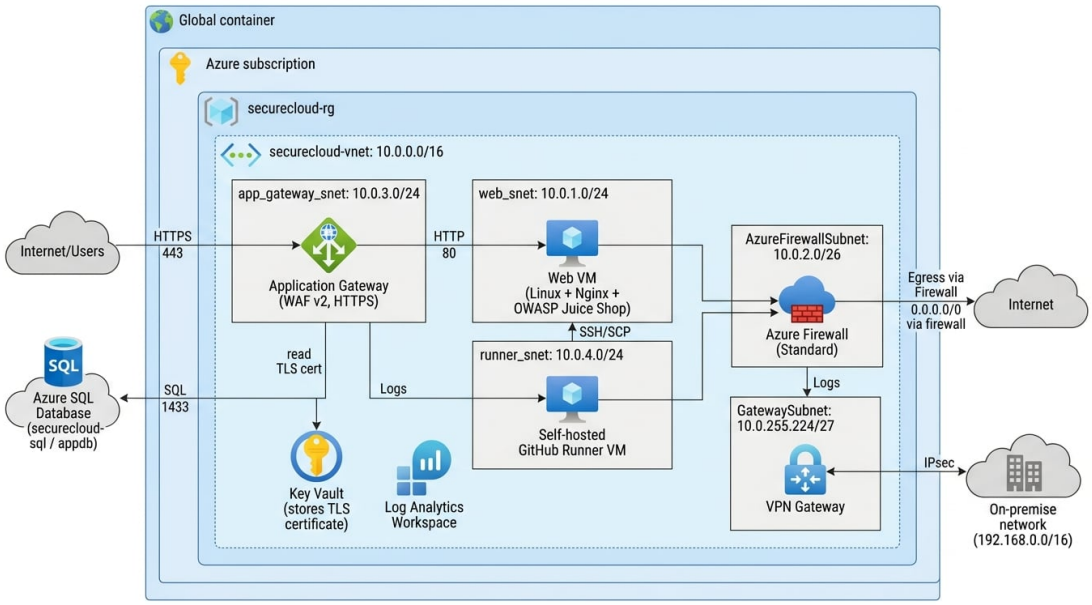


Figure II.2: *Architecture cible sécurisée adoptée intégrant tous les contrôles*

II.3 Installation et configuration

II.3.1 Environnement de mise en place

La mise en place de cette infrastructure a été réalisée sur deux machines de développement présentant les caractéristiques matérielles suivantes, permettant de supporter la charge des outils de virtualisation et d'infrastructure as code.

Le Tableau II.1 présente les postes de développement utilisés, dont la capacité matérielle est suffisante pour exécuter les outils de virtualisation et d'automatisation nécessaires au projet.

Caractéristique	PC 1	PC 2
Système d'exploitation	Windows 11	Windows 11
Processeur	Intel Core i5	AMD Ryzen 7
Mémoire RAM (Vive)	16 Go DDR5	32 Go DDR5
Stockage	512 Go SSD NVMe	1 To SSD NVMe

Table II.1: *Caractéristiques du matériel utilisé*

Pour la réalisation de ce projet, l'environnement logiciel a été recentré sur les plateformes essentielles. Le Tableau II.2 présente les outils retenus pour le développement, l'hébergement cloud et les tests de sécurité.

Outil / Technologie	Version	Rôle dans le projet
Windows 11	11	Système d'exploitation de développement local
Kali Linux	2023.x	Machine virtuelle d'attaque (simulation de menaces)
Microsoft Azure	Cloud	Hébergement de l'infrastructure cloud sécurisée
Visual Studio Code	Actuelle	Editeur de code principal

Table II.2: *Pile logicielle et outils utilisés*

II.3.2 Installation

Pour entamer l'automatisation de notre infrastructure, l'installation des outils de base sur nos machines de développement a été une étape primordiale. Nous avons procédé à l'installation de **Terraform** [2] pour la gestion de l'infrastructure en tant que code, ainsi que l'utilitaire **Azure CLI** [1] pour permettre l'authentification et l'interaction avec notre environnement Microsoft Azure directement depuis le terminal.

L'initialisation d'Azure CLI et l'authentification associée se sont déroulées avec succès, garantissant le couplage entre notre environnement local et notre souscription Cloud.

On retrouve ce résultat à la Figure II.3.

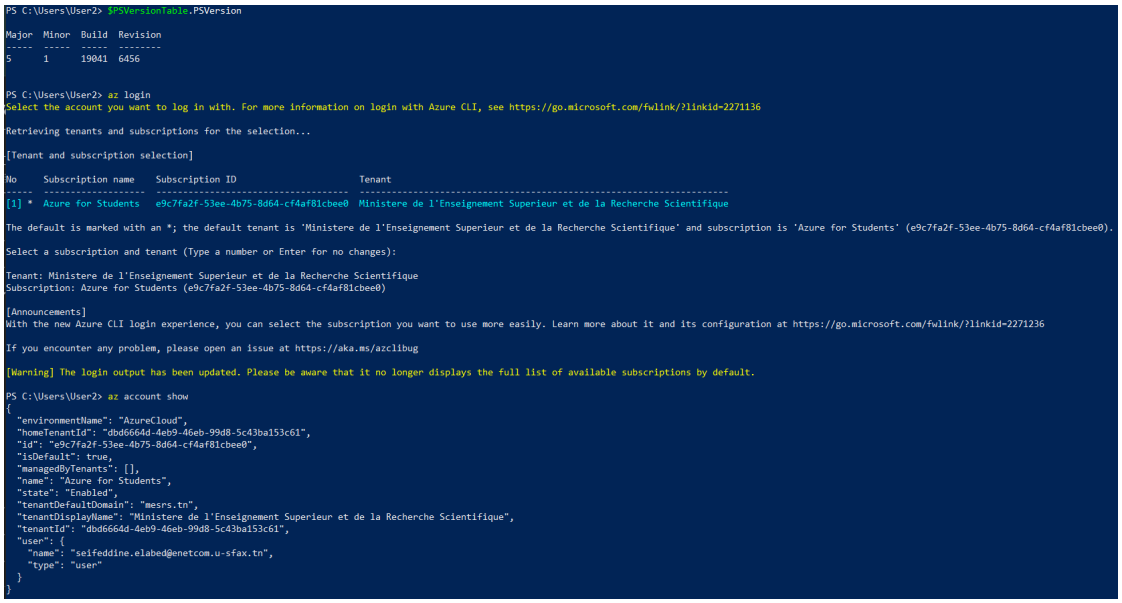


Figure II.3: *Initialisation et authentification Azure CLI*

II.3.3 Configuration

L'objectif de cette section est de formaliser les étapes clés de la configuration de l'environnement Cloud. L'approche Infrastructure as Code (IaC) adoptée garantit une reproductibilité parfaite et une gestion stricte des dépendances. Notre demarche reelle s'articule autour des points suivants :

II.3.3.1 Préparation et validation Terraform

La conception du code s'est fortement appuyée sur une structure modulaire visant à maximiser la réutilisation et la clarté du code. Nous avons distingué avec précision le routage `main` appelant de manière logique les différents sous-dossiers `modules` responsables chacun d'un composant spécifique (Réseau, Sécurité, Application). L'exploitation systématique des variables en entrée et des fichiers `outputs` a permis un débogage rapide. Cette vérification interne à l'outil (`terraform validate`) et la génération du plan d'exécution (`terraform plan`) ont confirmé la cohérence de l'architecture.

La Figure II.4 illustre ce point.

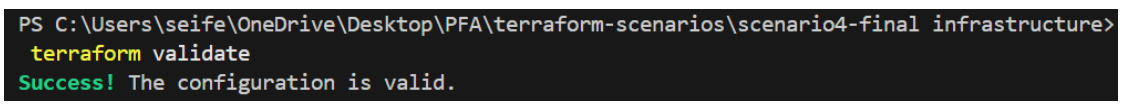


Figure II.4: *Préparation et validation Terraform*

II.3.3.2 Provisionnement des ressources

L'initiation du déploiement s'est traduite par une construction progressive respectant la topologie en couches. La commande d'application a interprété automatiquement l'ordre de création nécessaire, tel que l'établissement fondamental du réseau virtuel avant d'instancier les passerelles d'accès.

La Figure II.5 montre l'illustration correspondante.

```
PS C:\Users\seife\OneDrive\Desktop\PFA\terraform-scenarios\scenario4-final infrastructure>
 terraform apply -auto-approve
```

Figure II.5: *Provisionnement des ressources via Terraform*

II.3.3.3 Création de pipeline de déploiement avec GitHub Actions

Plutôt que d'employer une connexion classique en SSH ou FTP, le déploiement de la couche applicative est géré via un pipeline CI/CD structuré par un fichier descriptif `.yml` (**GitHub Actions**) à la racine du dépôt de code. Ce fichier dicte au runner distant l'authentification silencieuse à Azure via des identifiants stockés secrètement, et l'orchestration logicielle.

Voir la Figure II.6 pour l'illustration correspondante.

```yaml
name: Deploy OWASP Juice Shop to VM

on:
  workflow_dispatch:
    inputs:
      image_tag:
        description: "OWASP Juice Shop image tag"
        required: true
        default: "latest"

jobs:
  deploy:
    runs-on: [self-hosted, linux, x64, vpn]

    env:
      VM_HOST: ${{ vars.VM_HOST || '10.0.1.4' }}
      APP_HEALTH_URL: ${{ vars.APP_HEALTH_URL }}
      JUICE_IMAGE: ${{ vars.JUICE_IMAGE || 'bkimminich/juice-shop' }}
      JUICE_IMAGE_TAG: ${{ github.event.inputs.image_tag || 'latest' }}
```

Figure II.6: *Création de pipeline de déploiement avec GitHub Actions (YAML)*

II.3.3.4 Configuration de la VM avec Docker pour le déploiement

L'application cible (Juice Shop) [8] a été encapsulée dans un conteneur **Docker** [6]. Le pipeline déploie automatiquement cette instance encapsulée sur l'environnement cible VM Runner à chaque modification du code source.

II.3.3.5 Configuration DNS et HTTPS

Un nom de domaine légitime a été configuré pour pointer de manière transparente sur l'adresse IP publique de l'Application Gateway. Concernant la politique de chiffrement, nous avons procédé à l'importation de notre certificat SSL valide directement au sein de l'**Azure Key Vault**, assurant un chiffrement inaltérable des données en transit. Les étapes détaillées d'acquisition du domaine via Namecheap [9] et de configuration HTTPS sont documentées en annexe.

II.3.3.6 Configuration VPN

La configuration du VPN a été intégralement automatisée via notre code Terraform, assurant la création d'une passerelle réseau virtuelle dédiée au sein d'un sous-réseau spécifique (`GatewaySubnet`) en mode Point-to-Site (P2S) avec protocole OpenVPN. L'authentification repose sur un échange de certificats (Root et Client). Les règles de sécurité (NSG) ont été ajustées pour autoriser le flux d'administration.

On retrouve ce résultat à la Figure II.7.

```
vpn_client_configuration {
  address_space = [var.vpn_client_address_pool]

  vpn_client_protocols = ["OpenVPN"]

  vpn_auth_types = ["AAD"]

  aad_tenant    = "https://login.microsoftonline.com/${data.a
  aad_audience = "41b23e61-6c1e-4545-b367-cd054e0ed4b4"
  aad_issuer    = "https://sts.windows.net/${data.azurerm_cli
}
```

Figure II.7: *Configuration - Bloc Terraform client P2S*

II.4 Resultats

Suite à la configuration de notre infrastructure, nous presentons ci-dessous les resultats majeurs validant le bon fonctionnement de notre deploiement. Ces resultats illustrent la concretisation de nos scripts IaC et de nos modules d'automatisation.

Deploiement effectif de l'infrastructure

L'execution de la commande d'application Terraform a abouti à une convergence totale et sans erreur vers l'état cible, attestant de la stabilite de notre chaine IaC sur Azure. La Figure II.8 illustre ce point.

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

application_gateway_https_url = "https://20.111.42.85"
application_gateway_public_ip = "20.111.42.85"
resource_group_name = "securecloud-rg"
sql_server_fqdn = "securecloud-sql.database.windows.net"
vnet_id = "/subscriptions/e9c7fa2f-53ee-4b75-8d64-cf4af81cbee0/resourceGroups/securecloud-
rg/providers/Microsoft.Network/virtualNetworks/securecloud-vnet"
vpn_gateway_public_ip = "20.111.30.114"
web_vm_private_ip = "10.0.1.4"
```

Figure II.8: *Resultat - Fin de déploiement Terraform réussi*

Cette capture confirme dans *Azure Portal* la présence effective du groupe de ressources et des composants déployés, validant la réussite opérationnelle de l'infrastructure provisionnée.

La Figure II.9 montre l'illustration correspondante.

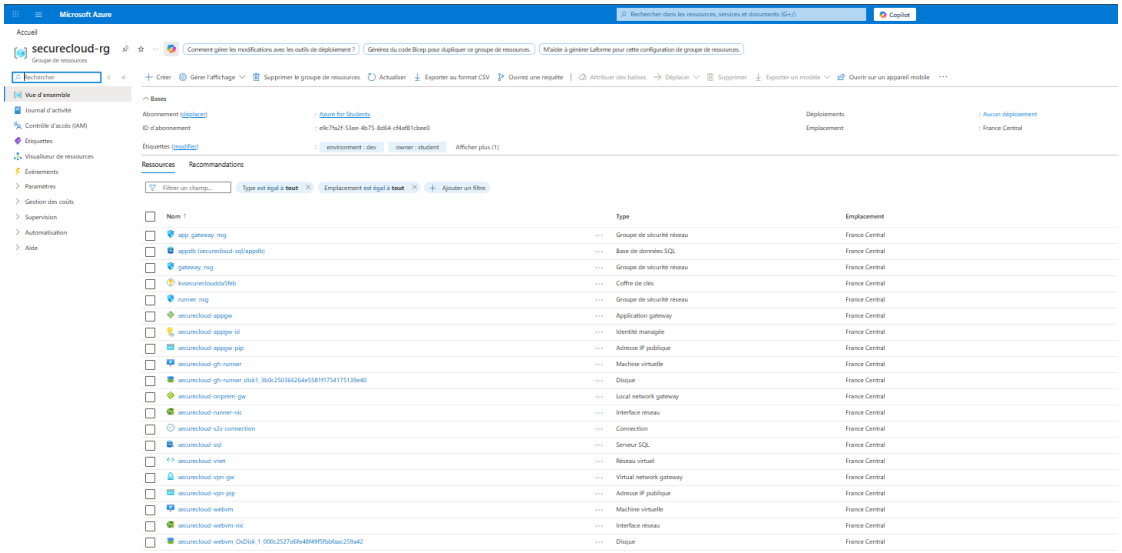


Figure II.9: *Resultat - Preuve de déploiement dans Azure Portal*

Execution du workflow CI/CD

Le pipeline GitHub Actions [7] s'est execute avec succes (liste verte), confirmant la reproductibilite de la publication applicative Docker sans requerir d'acces administrateur humain sur le serveur. Voir la Figure II.10 pour l'illustration correspondante.

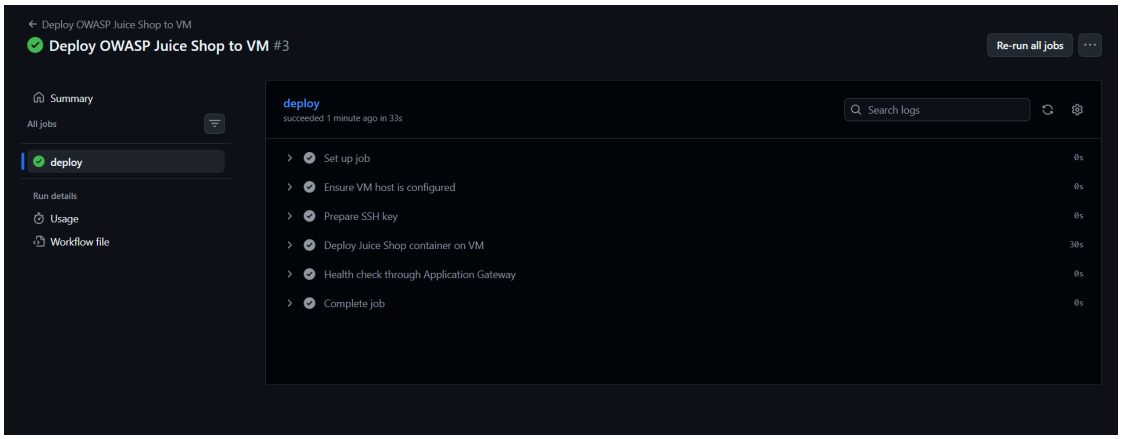


Figure II.10: *Resultat - Execution validee du workflow GitHub Actions*

Resolution DNS et domaine securise

La configuration reseau a ete actee par la propagation des enregistrements DNS pointant avec succes vers l'Azure Application Gateway. On retrouve ce résultat à la Figure II.11.

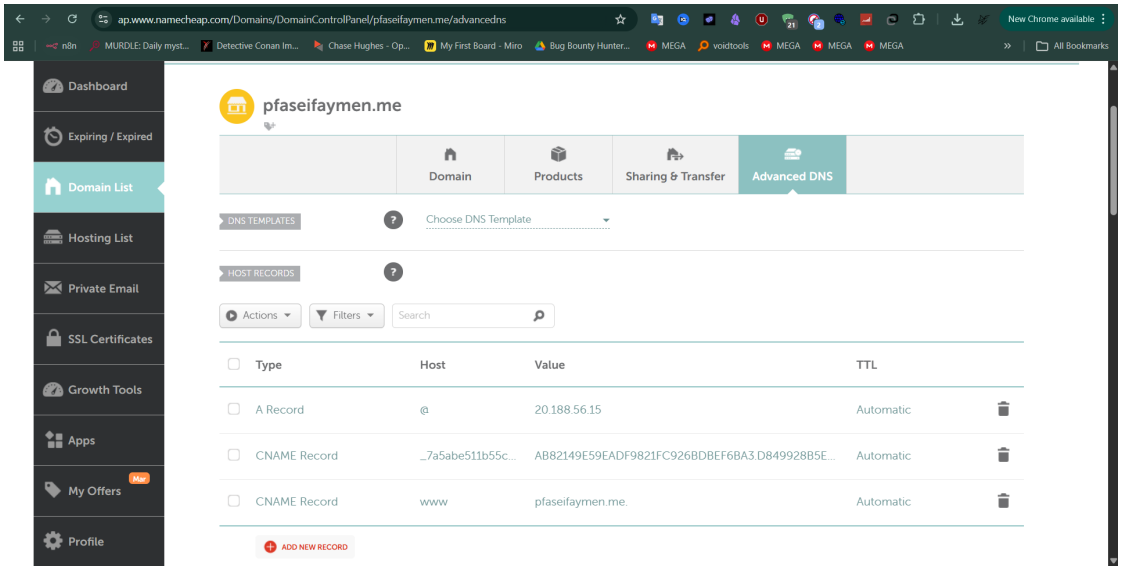


Figure II.11: *Resultat - Enregistrements de validation de domaine*

Acces d'administration via VPN

Le test de connexion Point-to-Site a abouti, attribuant une IP locale a l'administrateur et permettant l'ouverture d'une session distante (SSH) vers les ressources privees en toute securite. La Figure II.12 illustre ce point.

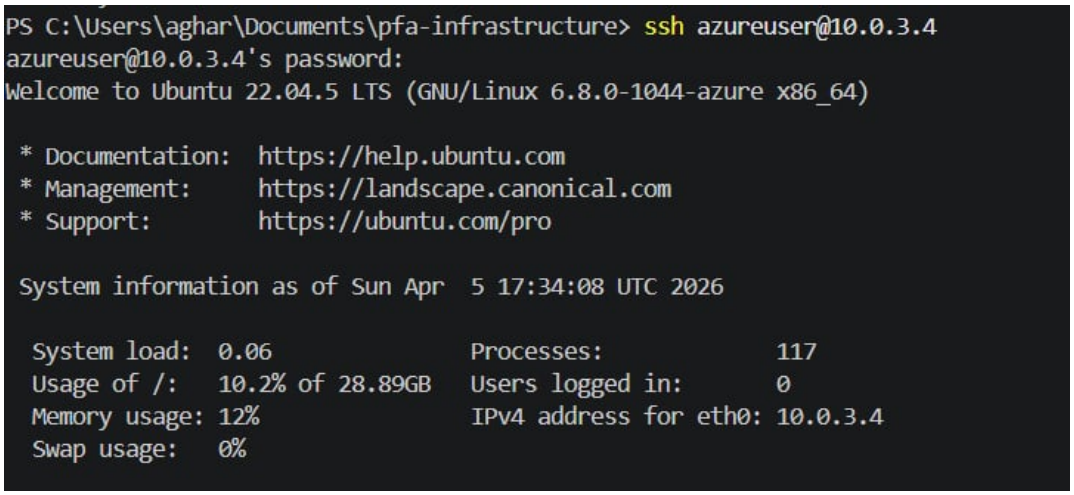


Figure II.12: *Resultat - Preuve de connexion SSH 1 via le tunnel VPN*

II.5 Test avec les attaques

Afin de valider pragmatiquement notre architecture orientée *Zero Trust*, nous avons défini un protocole d'évaluation offensif rigoureux ciblant l'infrastructure déployée. La méthodologie appliquée repose sur l'exécution d'attaques depuis un environnement Kali Linux distant, d'abord de manière manuelle pour cibler précisément les surfaces, puis automatisée pour éprouver la robustesse sous charge. L'objectif est d'observer les rejets par le WAF ou le pare-feu et d'appliquer une boucle de remédiation en cas de détection partielle.

II.5.1 Attaques manuelles

Les attaques manuelles ont permis d'évaluer le comportement de notre pare-feu applicatif (WAF) face à des requêtes malveillantes précisément forgées (Injections SQL et failles XSS).

a) Attaque par Injection SQL (SQLi)

Nous avons injecté manuellement une requête SQL (par exemple `' OR 1=1 -`) dans les formulaires d'authentification de l'application web pour tenter de contourner la logique de connexion. Dès l'envoi du *payload*, la requête a été interceptée par l'Azure Application Gateway. L'analyse des journaux du WAF Azure confirme formellement que la signature SQLi a déclenché une règle de blocage explicite (Mode Prévention).

La Figure II.13 montre l'illustration correspondante.

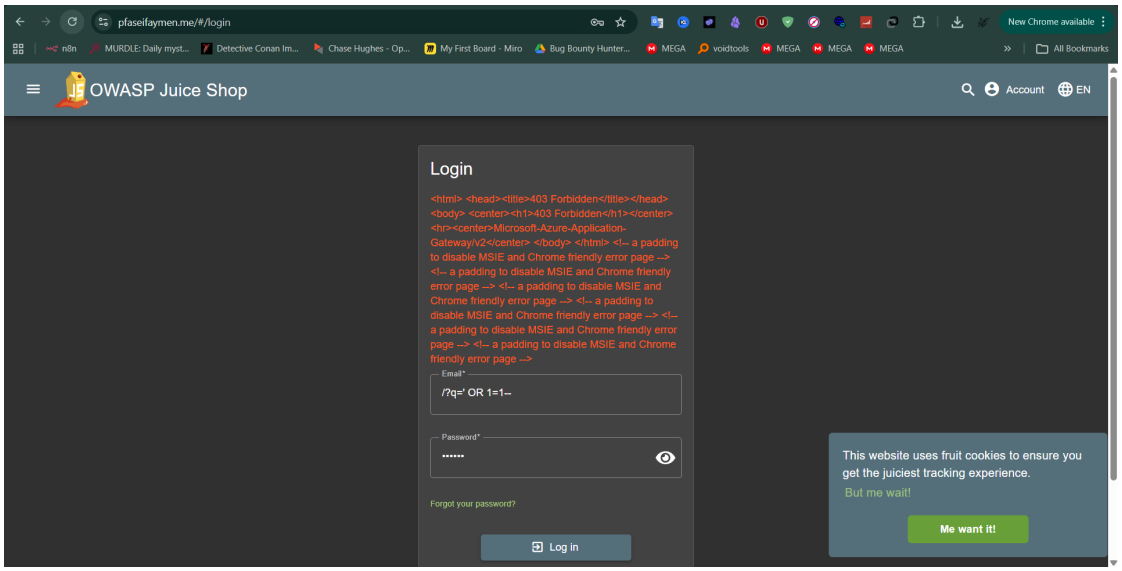


Figure II.13: *SQLi - Soumission du payload d'attaque*

Description du journal WAF : le journal confirme la détection d'une signature d'injection SQL, le déclenchement de la règle de sécurité associée et le blocage immédiat de la requête en mode prévention.

Voir la Figure II.14 pour l'illustration correspondante.

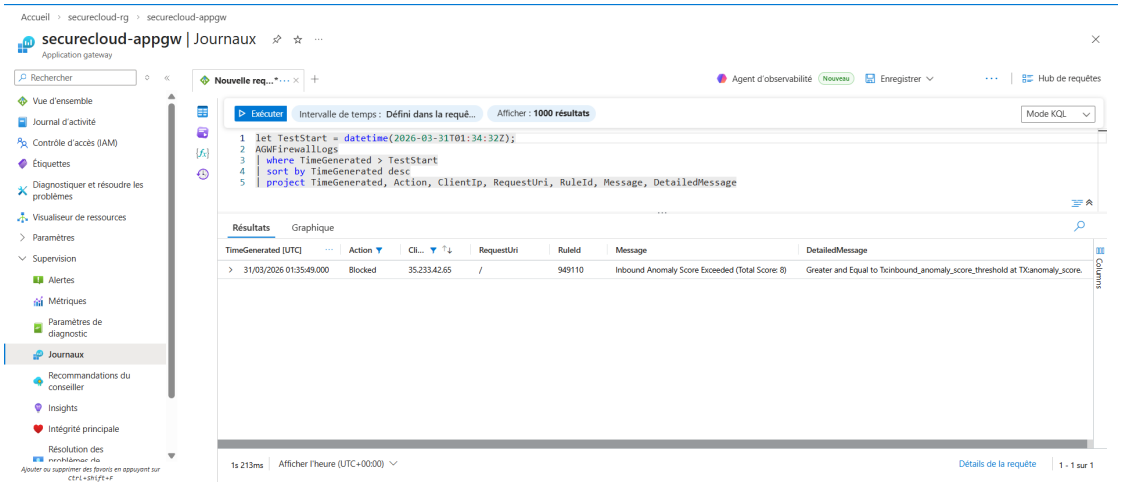


Figure II.14: *SQLi - Journaux WAF témoignant du blocage (Matched Rule)*

b) Attaque de type Cross-Site Scripting (XSS)

La seconde tentative consistait à insérer un script JavaScript malveillant (`<iframe src="javascript:alert('HACKED')">`) dans la barre de recherche ou un champ de commentaire de l'application. Cette méthode vise classiquement à dérober les sessions d'utilisateurs légitimes. Dans ce scénario, le WAF n'a pas révélé l'attaque, ce qui met en évidence une faille de sécurité nécessitant un durcissement des règles.

On retrouve ce résultat à la Figure II.15.

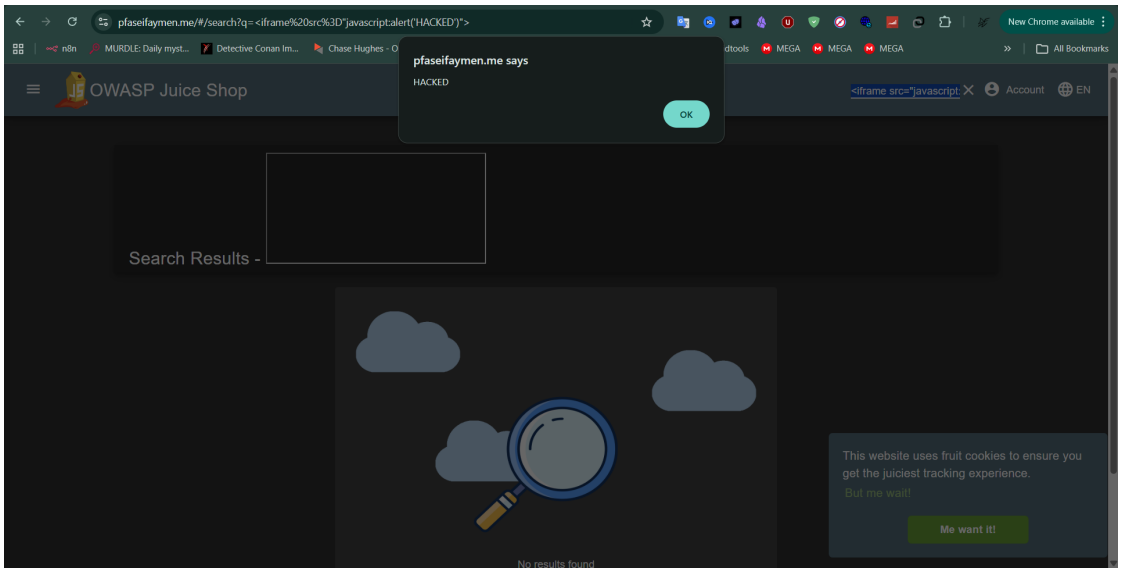


Figure II.15: *XSS - Soumission du payload d'attaque*

Description du journal WAF : les événements ne montrent pas d'identification explicite du motif XSS, ni de blocage effectif de la requête. Cette absence d'interception confirme une lacune de détection sur ce vecteur d'attaque.

La Figure II.16 illustre ce point.

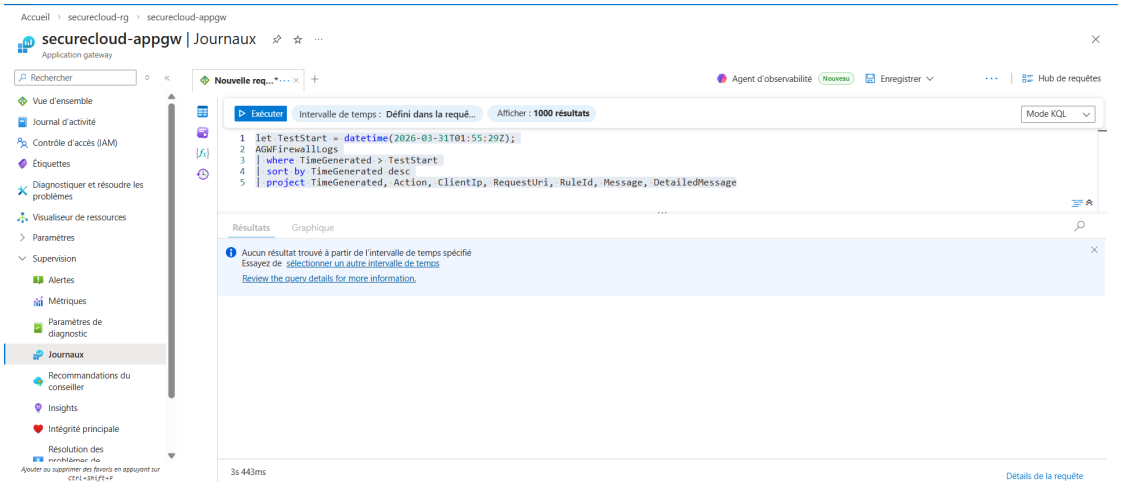


Figure II.16: *XSS - Journaux WAF n'interceptant pas le vecteur d'attaque*

II.5.2 Attaques automatisées

Afin d'évaluer la résilience de notre architecture face à des menaces industrielles, nous avons mobilisé une suite d'outils d'exploitation robotisés. L'objectif de cette campagne est d'éprouver la robustesse du pare-feu applicatif face à un volume élevé de requêtes malveillantes diversifiées.

II.5.2.1 Attaque automatisée 1: Balayage initial (Nikto)

Nous avons utilisé l'outil de balayage de vulnérabilités **Nikto** [5]. Dès le déclenchement de la commande, le trafic massif et non standard généré par Nikto a sollicité les mécanismes de sécurité de bordure. L'analyse des journaux du WAF démontre que l'Azure Application Gateway a intercepté et bloqué l'intégralité du balayage, rejetant chaque tentative d'énumération par des codes de réponse HTTP 403.

La Figure II.17 montre l'illustration correspondante.

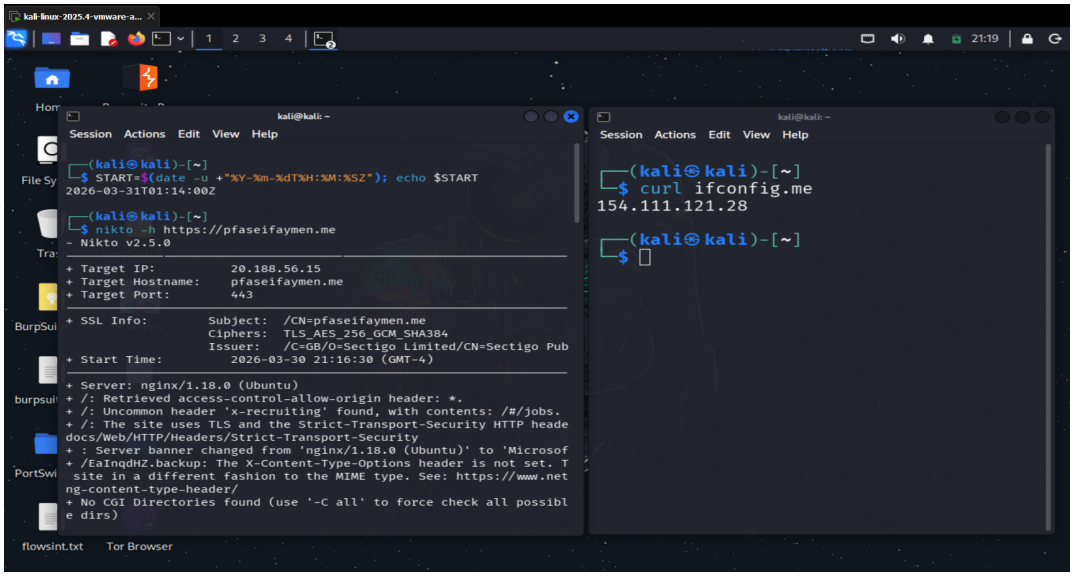


Figure II.17: *Automatisation - Exécution du balayage Nikto*

Description du journal WAF : les traces montrent une série de requêtes anormales détectées durant le scan Nikto, suivie de réponses de blocage (HTTP 403) sur les tentatives d'énumération.

Voir la Figure II.18 pour l'illustration correspondante.

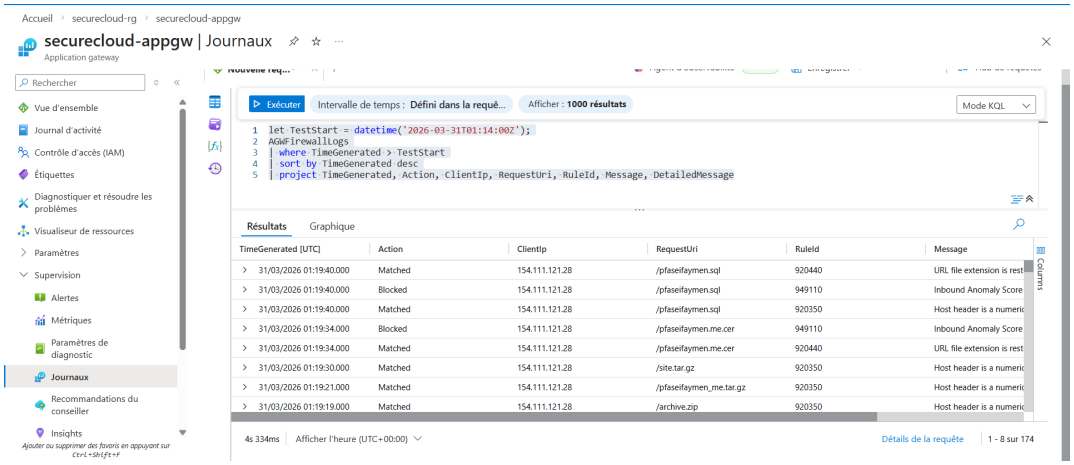


Figure II.18: *Automatisation - Journaux interceptant le balayage Nikto*

II.5.2.2 Attaque automatisée 2: SQL Injection (SQLMap)

Nous avons poursuivi avec **SQLMap** [10], un outil puissant d'analyse et d'exploitation automatisée ciblant spécifiquement la compromission des bases de données. Générant de multiples combinaisons d'injection très condensées, l'outil a systématiquement déclenché les seuils d'alerte de l'OWASP Core Rule Set. Les logs stipulent très clairement la reconnaissance des vecteurs SQL et leur rejet massif en prévention.

On retrouve ce résultat à la Figure II.19.

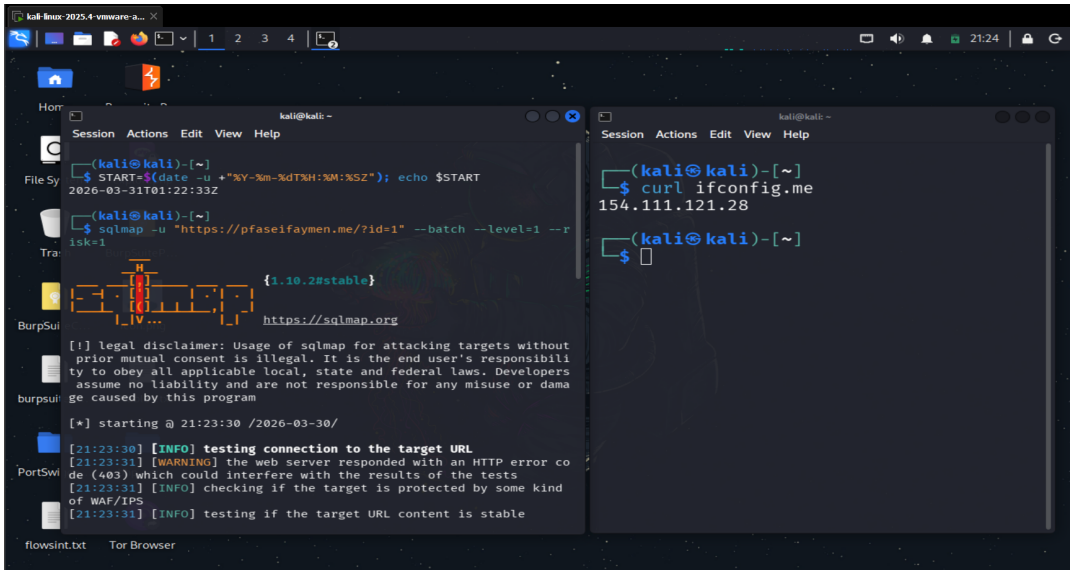


Figure II.19: *Automatisation - Exécution de SQLMap*

Description du journal WAF : le journal met en évidence des patterns d'injection SQL détectés en rafale, la correspondance avec les règles CRS et le rejet systématique des charges envoyées.

La Figure II.20 illustre ce point.

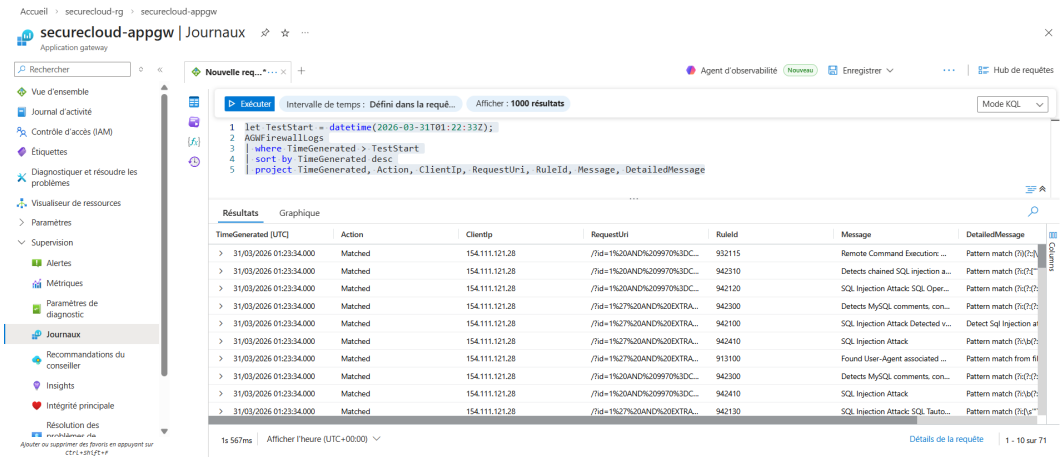


Figure II.20: *Automatisation - Journaux confirmant le blocage massif de SQLMap*

II.5.2.3 Attaque automatisée 3: Fuzzing (Wfuzz)

Pour finaliser ce stress-test, le fuzzer **Wfuzz** [11] a été invoqué afin d'employer des techniques de force brute sur les requêtes Web. L'outil a systématiquement substitué des paramètres de l'URL par de larges dictionnaires (bruteforcing directory, XSS payload). Là encore, l'analyse télémétrique au niveau du pare-feu applicatif confirme la reconnaissance pertinente de la charge virale d'obfuscation et de directory traversal, entraînant le bannissement du trafic malveillant.

La Figure II.21 montre l'illustration correspondante.

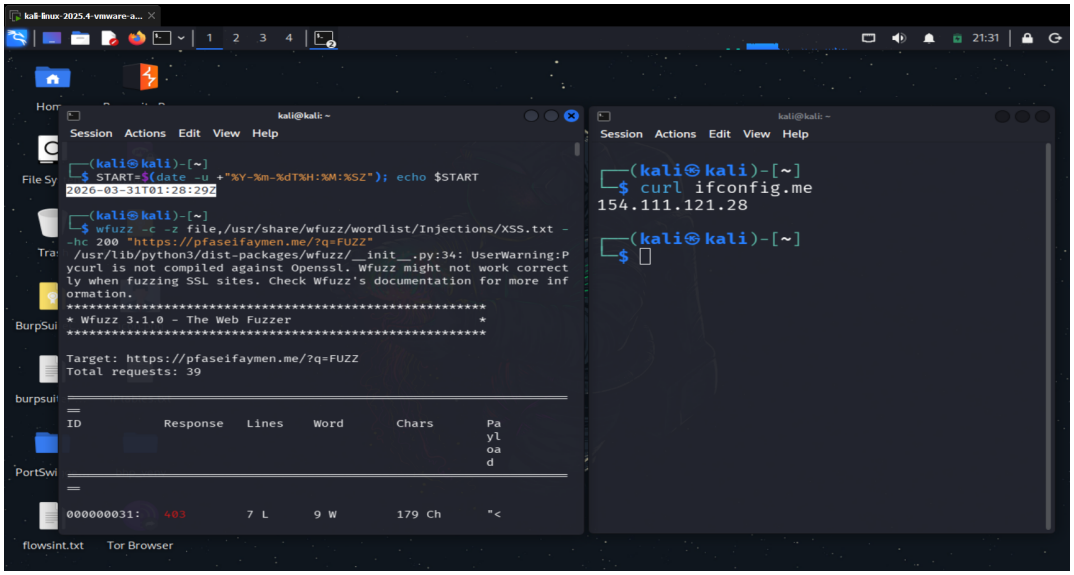


Figure II.21: *Automatisation - Exécution de Wfuzz*

Description du journal WAF : les logs indiquent la détection de requêtes de fuzzing et de motifs suspects (obfuscation/traversal), puis le blocage des tentatives selon la politique active.

Voir la Figure II.22 pour l'illustration correspondante.

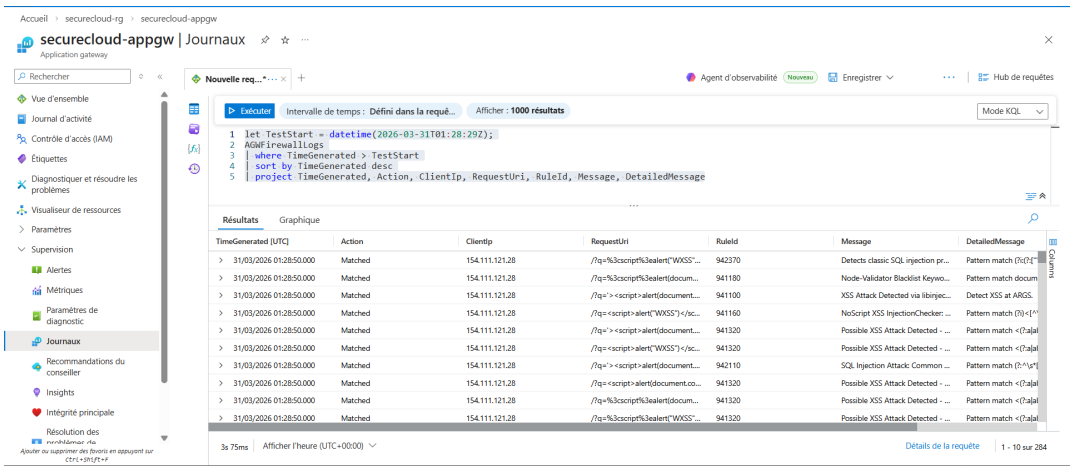


Figure II.22: *Automatisation - Journaux validant l'interception Wfuzz*

II.5.3 Analyse synthétique des résultats

L'évaluation des campagnes offensives démontre que notre architecture *Zero Trust* offre un très haut niveau de résilience périmétrique. L'efficacité s'avère totale contre les injections SQL, où le couple Application Gateway / WAF a rejeté systématiquement les requêtes avant même qu'elles n'atteignent le sous-réseau applicatif (Subnet VM Runner). L'absence complète de connectivité directe (bloquée par le NSG et Azure Firewall) confirme qu'une attaque réseau classique sur les ports standards depuis l'extérieur est devenue impossible.

Cependant, les tests ont révélé une couverture spatiale imparfaite concernant certaines déclinaisons très obfusquées d'attaques XSS. Bien que les signatures classiques fussent repoussées, des payloads encodés différemment ont réussi un "bypass" partiel des règles par défaut strictes. Ce constat souligne qu'une plateforme sécurisée ne repose pas de manière magique sur une solution unique, mais exige une calibration humaine continue et itérative des seuils d'alerte, prouvant tout l'intérêt de la supervision active.

II.6 Problèmes résolus & remédiations

Notre implémentation logicielle couplée aux évaluations intrusives a exposé ou corrigé des défaillances critiques au sein de l'environnement :

- **Problème 1 : Derive de configuration et incoherence entre cycles -** L'incohérence systématique des ressources créées manuellement a été balayée par l'usage asynchrone du code déclaratif Terraform en intégration continue, standardisant la posture défensive de Dev à Prod.
- **Problème 2 : Exposition reseau excessive -** L'absence de filtrage applicatif a été remédiée par le déploiement de l'Application Gateway WAF isolant les machines internes d'un accès frontal direct.
- **Problème 3 : Manque de gouvernance des sorties egress -** Les fuites de données internes ont été endiguées au moyen d'un routage coercitif (UDR) redirigeant le trafic vers un concentrateur Azure Firewall qui bloque par défaut toute requête sortante impropre.
- **Problème 4 : Acces d'administration insuffisamment cloisonne -** Les ponts RDP/SSH publics ont été aveuglés au profit d'un VPN Azure point-à-site cryptographique, qui garantit aux administrateurs locaux un tunnel sans projection sur Internet.
- **Problème 5 : Couverture defensive inegale selon les attaques -** L'analyse des journaux des scans automatisés et fuzzing a soulevé la faillite partielle du Core Rule Set par défaut de l'OWASP sur certains payloads d'obfuscation XSS ou Path Traversal.

Pour contrecarrer le cas particulier induit par le Problème 5 (l'attaque XSS contournant partiellement le WAF), il a été nécessaire d'aller au-delà des règles par défaut ("Core Rule Set" d'OWASP). La logique de remédiation appliquée consistait à forger une règle personnalisée (*Custom Rule*) au niveau du profil de la stratégie WAF, forçant l'Azure Application Gateway à rejeter explicitement les expressions régulières complexes issues de ces scripts.

Le bloc de code déclaratif suivant illustre l'ajout de cette remédiation vitale dans notre configuration Terraform, forçant le rejet du trafic contenant ces anomalies de balises :

```
resource "azurerm_web_application_firewall_policy" "waf_policy" {
  ...
  custom_rules {
    name      = "BlockObfuscatedXSS"
    priority  = 100
    rule_type = "MatchRule"
    action    = "Block"
    match_conditions {
      match_variables {
        variable_name = "RequestUri"
      }
      operator           = "Regex"
      match_values       = ["(?:<script.*?>|javascript:)"]
      transforms         = ["Lowercase", "UrlDecode"]
    }
  }
}
```

Le diagramme ci-dessous consolide visuellement notre trajectoire globale en mettant en avant le fil conducteur de notre projet.

On retrouve ce résultat à la Figure II.23.

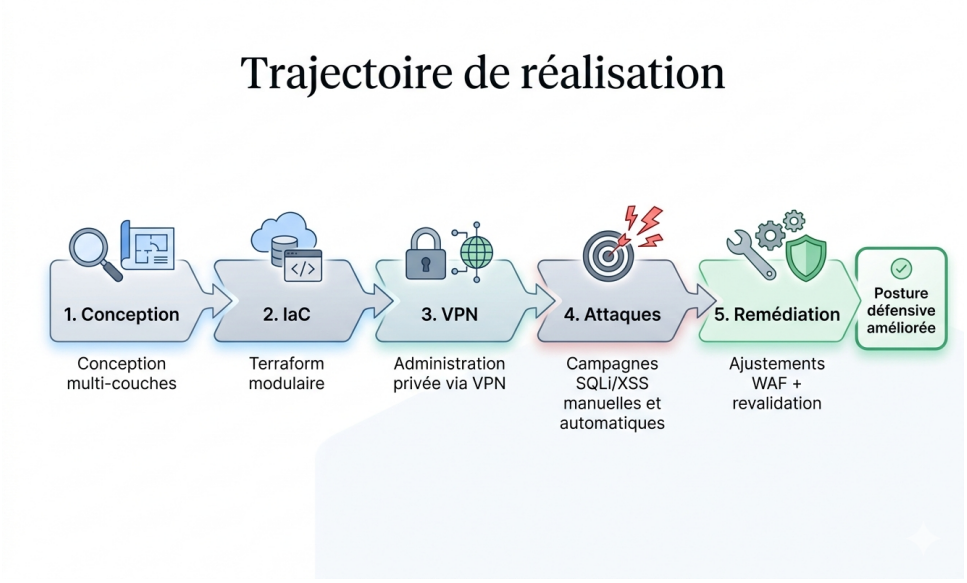


Figure II.23: *Diagramme - Trajectoire de réalisation de la remédiation*

L'application de ce code via l'approche IaC garantit que l'infrastructure corrigée se redéploie de façon identique et immédiate. Ce cycle court d' "observation-remédiation-validation" a permis d'élever considérablement la maturité défensive de l'architecture.

Le diagramme de maturité défensive illustre la transition d'un modèle traditionnel poreux vers un état maîtrisé et verrouillé, actant la valeur ajoutée finale de ce travail concret.

La Figure II.24 illustre ce point.

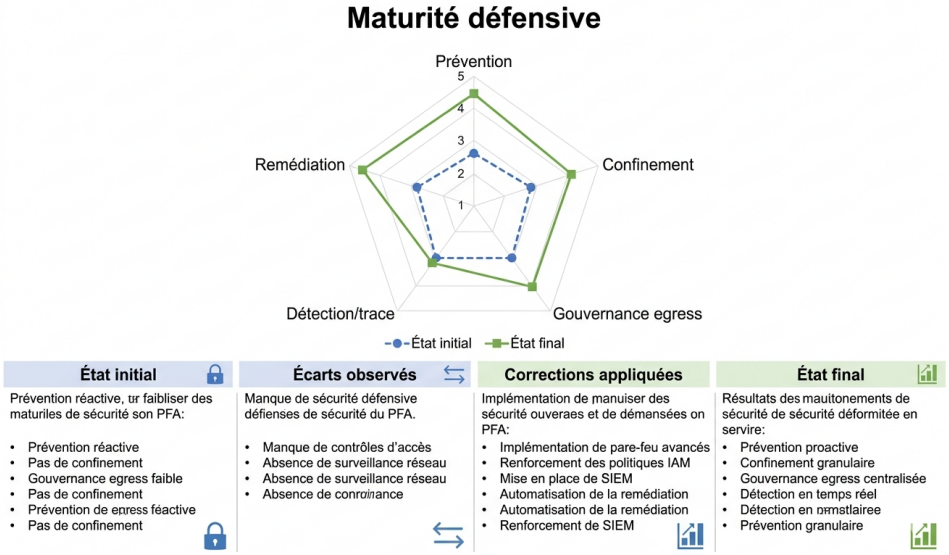


Figure II.24: *Diagramme - Maturité défensive de l'architecture finale*

II.7 Conclusion

Ce chapitre a permis de balayer l'ensemble du cycle d'ingénierie : configuration, preuve de marche, validation offensive, correction et revalidation de l'architecture cible. La densité des preuves présentées confirme la valeur ajoutée du projet et sa cohérence technique, validant in fine l'hermétisme de l'infrastructure face aux menaces cyber contemporaines.

CONCLUSION GENERALE

Ce projet de fin d'année avait pour objectif de concevoir, déployer et valider une architecture cloud sécurisée sur Azure, en s'appuyant sur une démarche d'ingénierie claire et reproductible. Le cœur du travail a été de transformer des objectifs de sécurité en choix techniques concrets : segmentation réseau, filtrage du trafic, protection applicative, accès d'administration privé et automatisation du déploiement. La mise en œuvre a été réalisée avec Terraform et organisée en modules, puis complétée par une chaîne CI/CD avec GitHub Actions afin de garantir la cohérence entre conception et exécution.

La validation ne s'est pas limitée à une simple vérification du déploiement ; elle a été prolongée par des tests d'attaque manuels et automatisés (SQLi, XSS, Nikto, SQLMap, Wfuzz), afin de mesurer l'efficacité réelle des mécanismes défensifs et d'identifier les écarts.

Les bénéfices du projet sont à la fois techniques et pédagogiques. Sur le plan technique, nous avons obtenu une infrastructure reproductible, plus stable et plus facile à faire évoluer grâce à l'approche IaC. Nous avons également renforcé la posture défensive globale : réduction de l'exposition directe, meilleur contrôle des flux entrants et sortants, sécurisation de l'administration via VPN Point-to-Site, et protection applicative au niveau de l'Application Gateway WAF.

Le projet a également mis en évidence l'intérêt d'une boucle continue « déploiement → test → correction → revalidation », qui améliore la fiabilité de l'architecture au fil des itérations. Sur le plan pédagogique, nous avons développé une compétence essentielle en ingénierie de la sécurité : relier chaque décision à un risque, documenter des preuves techniques et présenter un raisonnement exploitable par un tiers.

Comme tout projet appliqué, notre réalisation présente certaines limites. La première concerne la couverture des tests : même si les scénarios offensifs exécutés sont représentatifs, ils ne couvrent pas l'ensemble des menaces possibles ni toutes les variantes d'évasion. La deuxième limite est liée au contexte de PFA : le temps et les ressources ne permettent pas d'atteindre le niveau de durcissement d'un environnement de production critique, avec supervision 24/7, red team continue et audits réglementaires complets.

La troisième limite concerne la configuration défensive fine : certains cas (notamment des formes obfusquées d'attaques) exigent un ajustement itératif des règles WAF et une observation continue des journaux afin de maintenir un bon compromis entre sécurité et disponibilité. Enfin,

une partie des validations reste ponctuelle et devrait être davantage industrialisée dans des campagnes automatiques périodiques.

Nos perspectives d'amélioration sont donc claires. À court terme, nous pouvons enrichir les règles WAF (notamment via des règles personnalisées contextuelles), étendre les jeux de tests d'attaque et automatiser davantage la revalidation après chaque changement d'infrastructure ou d'application.

À moyen terme, l'intégration de mécanismes d'observabilité plus avancés (tableaux de bord de sécurité, corrélation d'événements, alerting structuré) permettrait une détection plus précoce et une réponse plus rapide. À plus long terme, l'architecture peut évoluer vers un cadre quasi production avec une gouvernance multi-environnements plus stricte, des politiques de conformité formalisées et une approche DevSecOps encore plus poussée dans les pipelines CI/CD.

En conclusion, le projet a rempli son objectif principal : démontrer qu'il est possible de construire une architecture cloud sécurisée de manière méthodique, de la valider par la preuve, puis de l'améliorer par remédiation. Au-delà des livrables techniques, il nous a apporté une méthode transférable pour de futurs projets : analyser le risque, concevoir des contrôles adaptés, automatiser le déploiement, tester en conditions réelles et corriger de façon traçable.

Cette combinaison entre rigueur technique, logique d'amélioration continue et lisibilité académique constitue la valeur centrale de notre travail.

BIBLIOGRAPHIE

[1] Microsoft Azure. **Azure Documentation et Services Cloud**. *En ligne*. Disponible sur: https://docs.microsoft.com/en-us/azure/ (consulte en 2024).

[2] HashiCorp. **Terraform Official Documentation**. *En ligne*. Disponible sur: https://www.terraform.io/docs/ (consulte en 2024).

[3] OWASP Foundation. **OWASP Top 10 2023 - Web Application Security Risks**. *En ligne*. Disponible sur: https://owasp.org/www-project-top-ten/ (consulte en 2024).

[4] Offensive Security. **Kali Linux - Penetration Testing Distribution**. *En ligne*. Disponible sur: https://www.kali.org/ (consulte en 2024).

[5] Sullo, David. **Nikto2 - Web Server Scanner**. *En ligne*. Disponible sur: https://www.cirt.net/Nikto2/ (consulte en 2024).

[6] Docker. **Docker - Containerization Platform**. *En ligne*. Disponible sur: https://www.docker.com/ (consulte en 2024).

[7] GitHub. **GitHub Actions - Workflow Automation and CI/CD**. *En ligne*. Disponible sur: https://github.com/features/actions (consulte en 2024).

[8] OWASP. **JuiceShop - Open Source Vulnerable Web Application**. *En ligne*. Disponible sur: https://owasp-juice.shop/ (consulte en 2024).

[9] Namecheap. **Domain Registration and SSL Certificates**. *En ligne*. Disponible sur: https://www.namecheap.com/ (consulte en 2024).

[10] Bernardo, Bernardo. **SQLMap - Automatic SQL Injection and Database Takeover Tool**. *En ligne*. Disponible sur: https://sqlmap.org/ (consulte en 2024).

[11] Xavi Mendez. **Wfuzz - Web Application Fuzzer**. *En ligne*. Disponible sur: https://wfuzz.readthedocs.io/ (consulte en 2024).

[12] Sectigo. **Public Server Authentication DV Certificate**. *En ligne*. Disponible sur: https://sectigo.com (consulte en 2024).

[13] Automat-IT. **Hack the OWASP Juice Shop application and protect it with AWS WAF - Part 2**. *En ligne*. Disponible sur: https://www.automat-it.com/blog/hack-the-owasp-juice-shop-application-and-protect-it-with-aws-waf-part-2/ (consulte en 2026).

ANNEXES

A.1 Integration et Configuration du VPN Point-to-Site

A.1.1 Etape 1 : Generation du package client

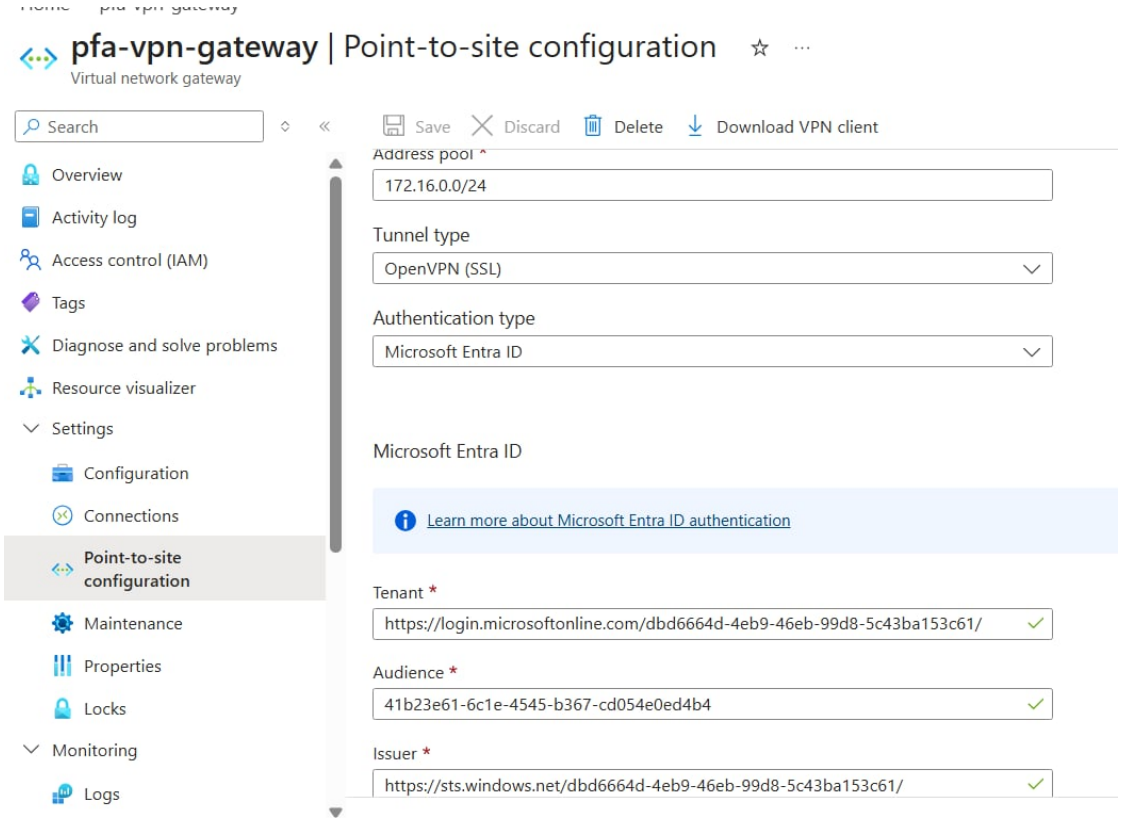


Figure A.1: *VPN client - Etape 1: generation du package*

Le package client P2S est genere depuis le portail Azure.

A.1.2 Etape 2 : Installation du client VPN

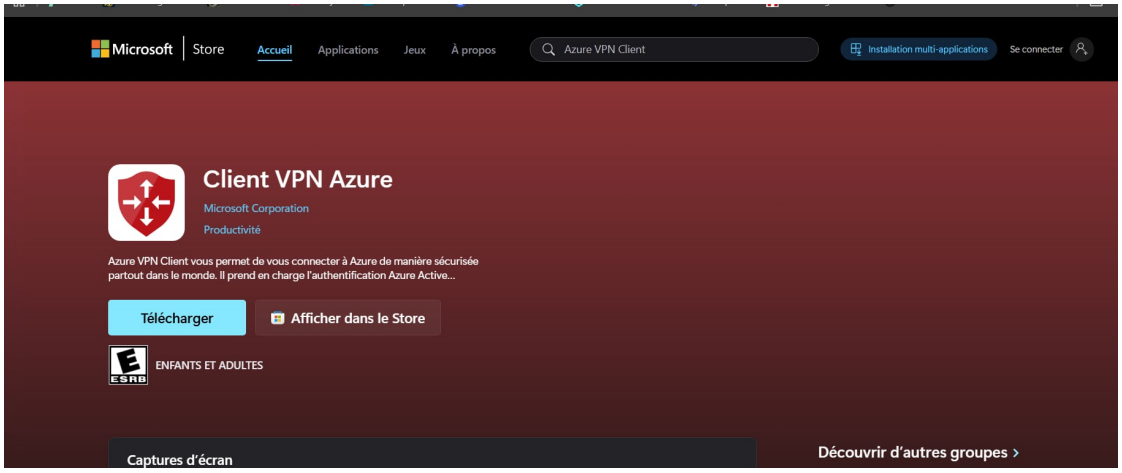


Figure A.2: *VPN client - Etape 2: installation Azure VPN Client*

Le client VPN est installe sur le poste d'administration.

A.1.3 Etape 3 : Preparation pour l'import du profil

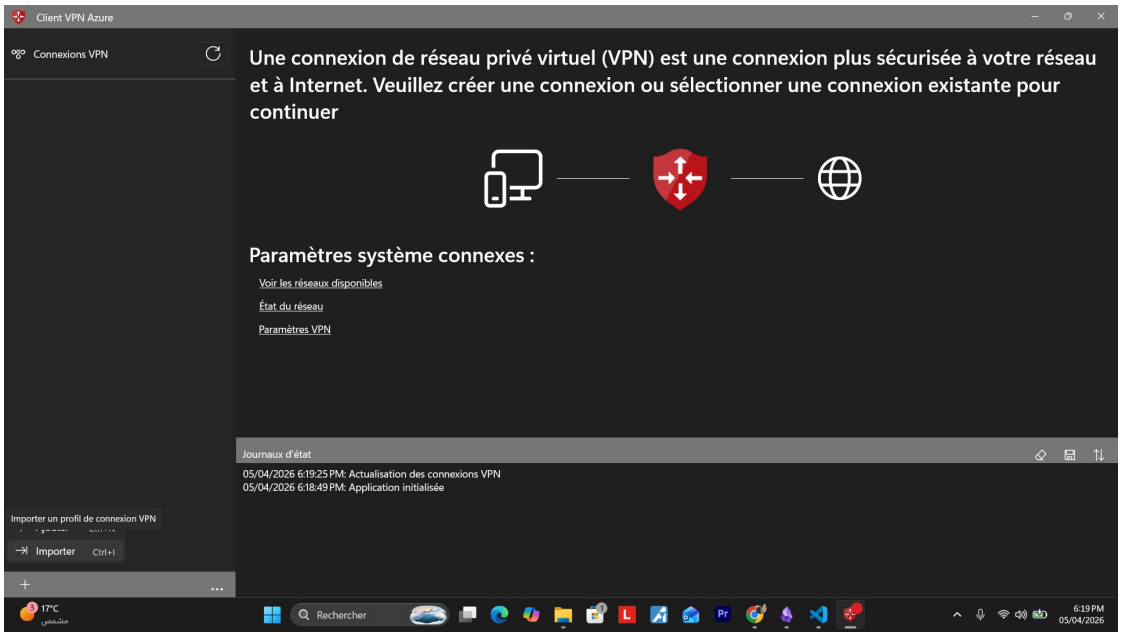


Figure A.3: *VPN client - Etape 3: interface et preparation import*

Cette etape prepare l'import du profil de connexion.

A.1.4 Etape 4 : Selection du fichier de configuration

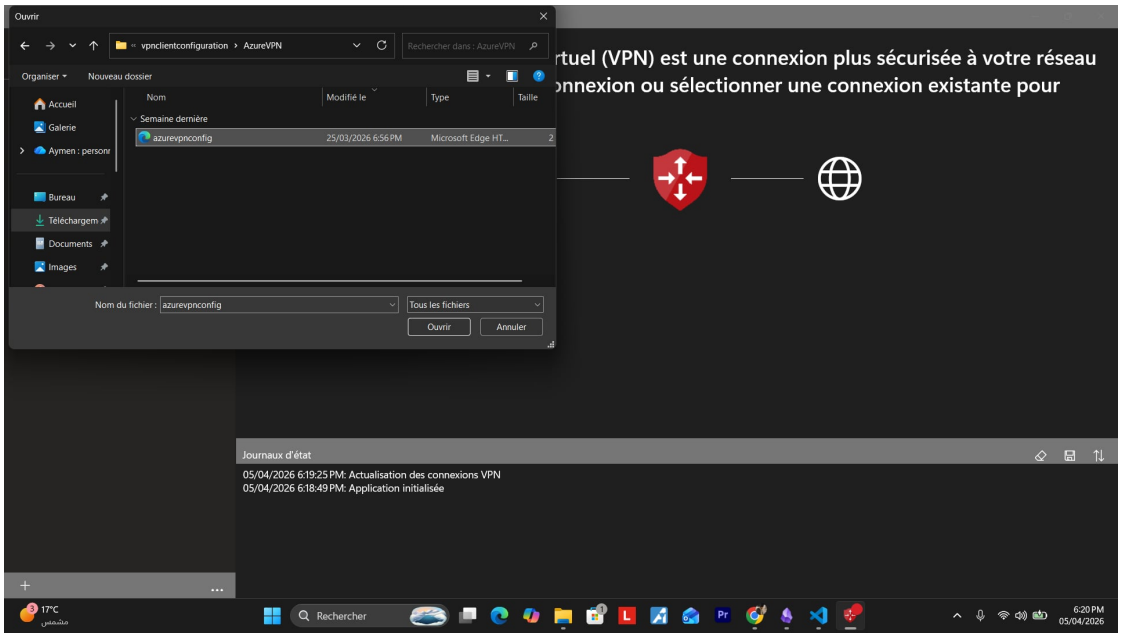


Figure A.4: *VPN client - Etape 4: selection azurevpnconfig.xml*

Le profil XML est choisi pour initialiser la configuration P2S.

A.1.5 Etape 5 : Import des parametres de connexion

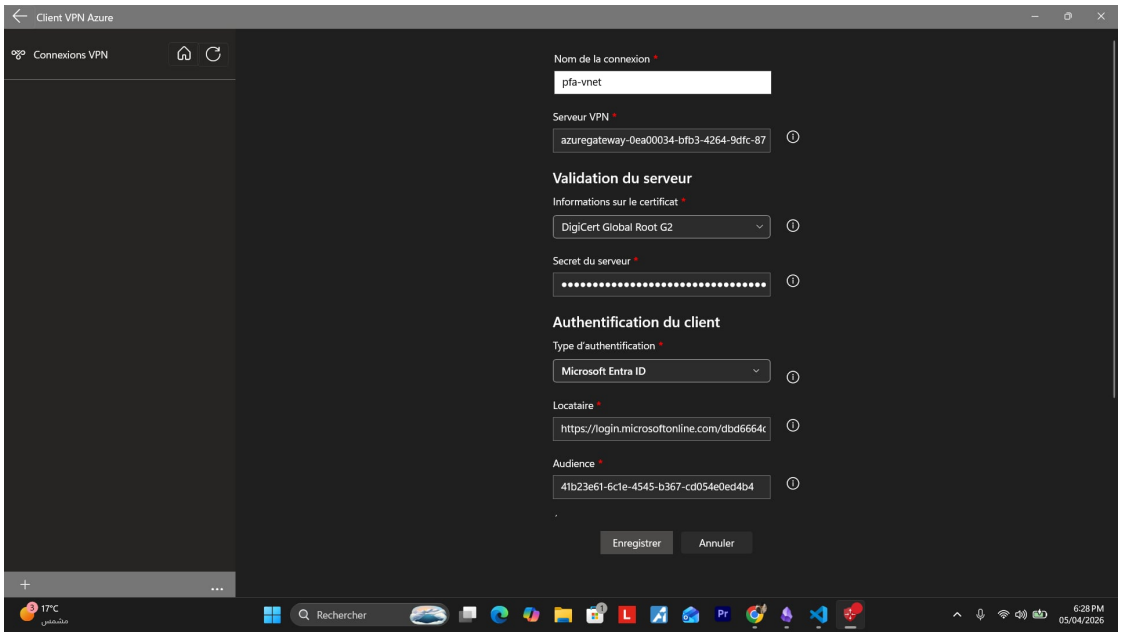


Figure A.5: *VPN client - Etape 5: parametres importes*

Les parametres serveur, authentification et locataire sont charges.

A.1.6 Etape 6 : Validation du profil importe

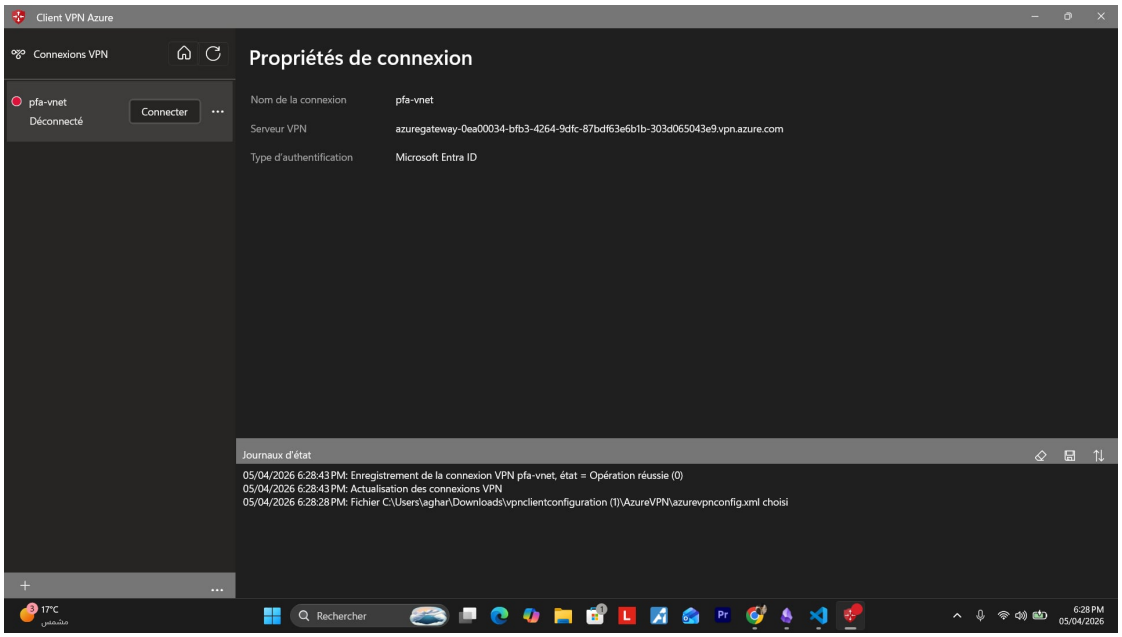


Figure A.6: *VPN client - Etape 6: profil importe*

Cette capture confirme la disponibilite du profil de connexion.

A.1.7 Etape 7 : Etablissement de la connexion VPN

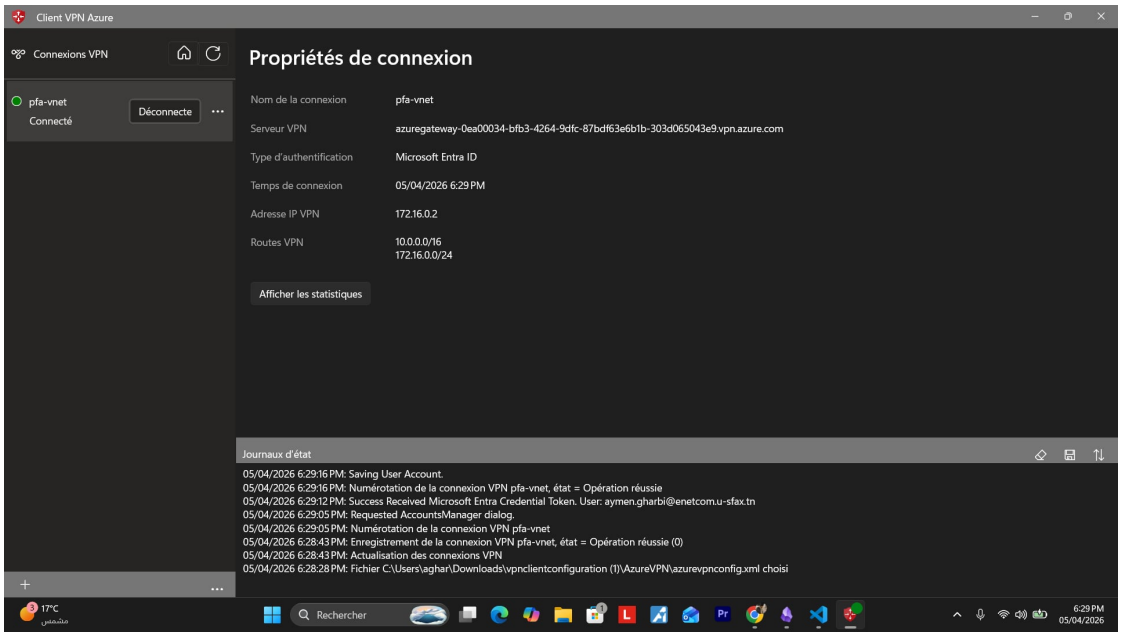


Figure A.7: *VPN client - Etape 7: connexion etablie*

Le tunnel VPN est actif et pret pour l'administration privee.

A.1.8 Etape 8 : Verification finale de la connexion

```
Carte PPP pfa-vnet :

   Suffixe DNS propre à la connexion. . . :
   Description. . . . . . . . . . . . . . : pfa-vnet
   Adresse physique . . . . . . . . . . . :
   DHCP activé. . . . . . . . . . . . . . : Non
   Configuration automatique activée. . . : Oui
   Adresse IPv4. . . . . . . . . . . . . .: 172.16.0.2(préféré)
   Masque de sous-réseau. . . . . . . . . : 255.255.255.255
   Passerelle par défaut. . . . . . . . . :
   NetBIOS sur Tcpip. . . . . . . . . . . : Activé
```

Figure A.8: *VPN client - Etape 8: verification finale*

Cette verification confirme la stabilite de la connexion client.

A.1.9 Configuration et verification des ressources VPN

La configuration complete de la passerelle VPN et des regles de securite associees est validee par les captures suivantes :

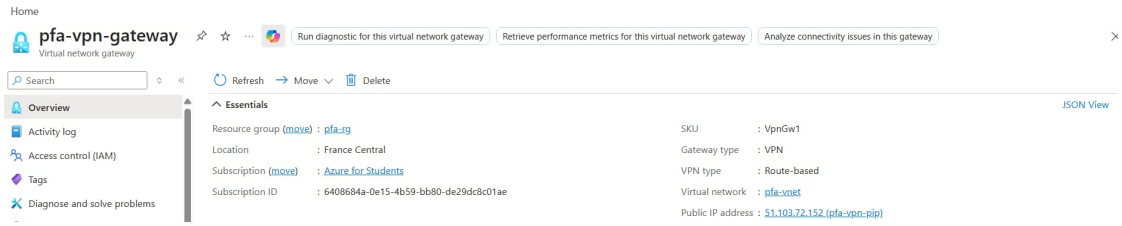


Figure A.9: *VPN - Gateway deploye dans Azure*

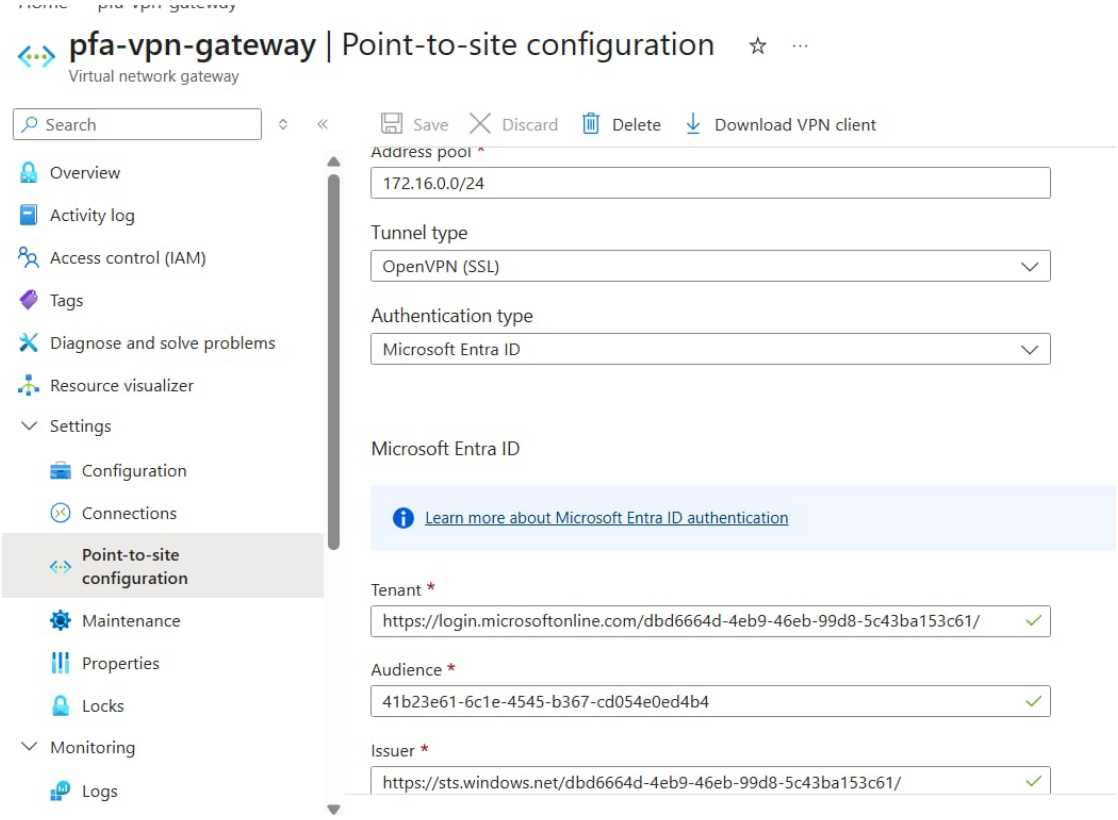


Figure A.10: *VPN - Parametres Point-to-Site OpenVPN*

```
security_rule {
  name                       = "Allow-HTTP-From-VPN"
  priority                   = 105
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "Tcp"
  source_port_range          = "*"
  destination_port_range     = "80"
  source_address_prefix      = "172.16.0.0/24"
  destination_address_prefix = "*"
}
```

Figure A.11: *VPN - Regle de securite NSG bloquante par defaut*

105	Allow-HTTP-From-VPN	80	Tcp	172.16.0.0/24	Any	Allow	
110	Allow-SSH-From-VPN	22	Tcp	172.16.0.0/24	Any	Allow	
4096	Deny-All-Inbound	Any	Any	Any	Any	Deny	

Figure A.12: *VPN - Verification des regles NSG dans le portail Azure*

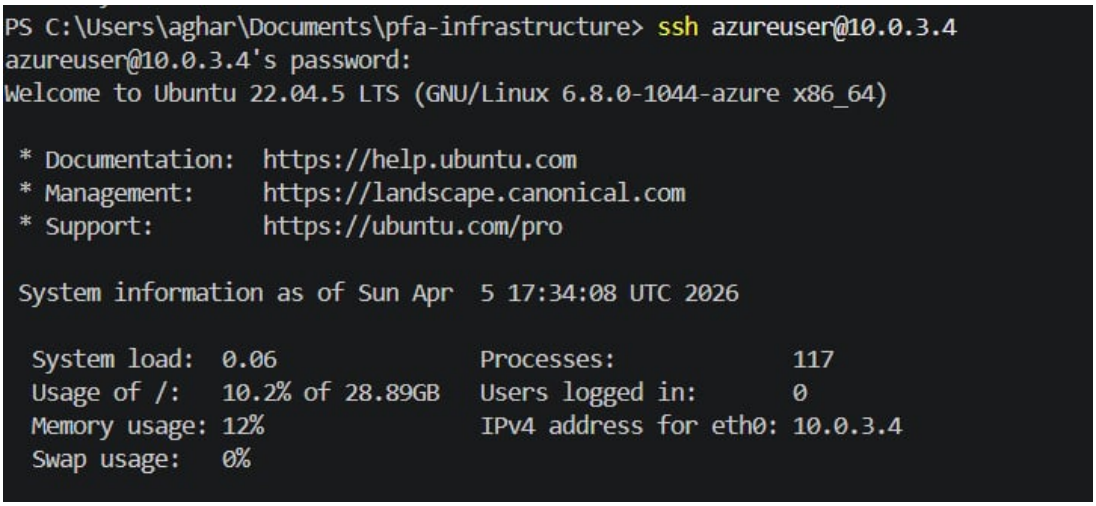


Figure A.13: *VPN - Preuve SSH 1 via tunnel*

A.2 Tracabilite des actifs images

La tracabilite des captures est maintenue dans le dossier *images* avec une nomenclature normalisee pour la compilation, tout en conservant les fichiers sources originaux.

A.3 Processus d'achat et configuration DNS / TLS via Namecheap

Afin de securiser de facon professionnelle l'exposition de notre application, il etait necessaire d'acquerir un nom de domaine legitime et un certificat SSL/TLS reconnu. Pour ce faire, nous avons exploite le **GitHub Student Developer Pack**, qui offre aux etudiants un acces gratuit a divers outils, incluant l'enregistrement de domaines via le registraire **Namecheap**.

La raison profonde de cette demarche depasse le simple aspect esthetique (remplacer une adresse IP par un nom comprehensible). L'objectif est d'assurer au client que le serveur auquel il se connecte est authentique et chiffre, tout en permettant a l'Application Gateway Azure de router les flux correctement sans lever d'alertes de securite au niveau du navigateur ("Connexion non securisee"). Le certificat SSL, une fois acquis sur Namecheap, a ete extrait et directement integre a l'Azure Key Vault pour deploiement.

A.3.1 Configuration initiale et preuves d'integration

Cette section regroupe les elements visuels complementaires attestant des jalons de configuration DNS et de l'integration HTTPS.

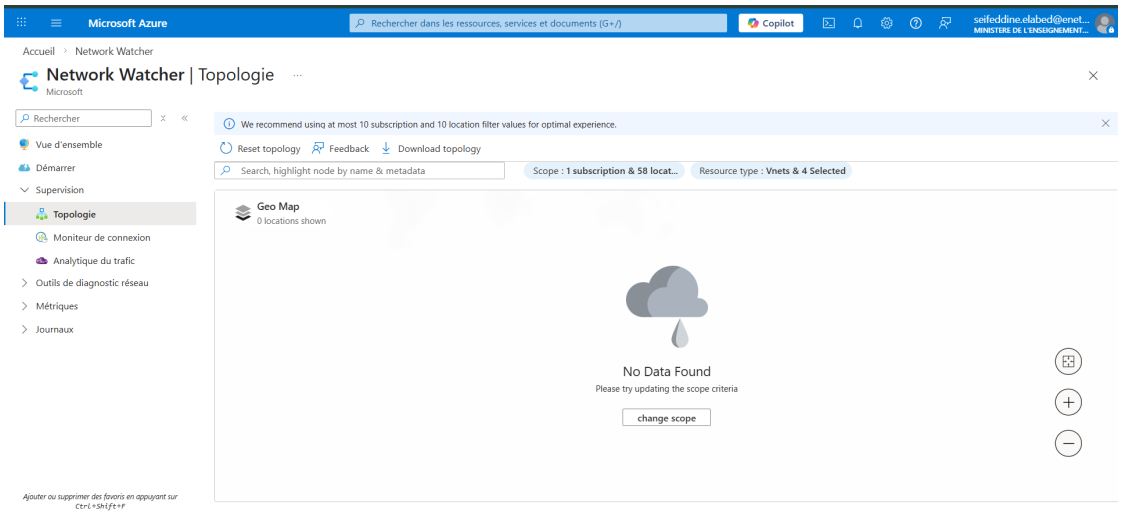


Figure A.14: *Application - Acces IP publique avant domaine (Alerte TLS)*

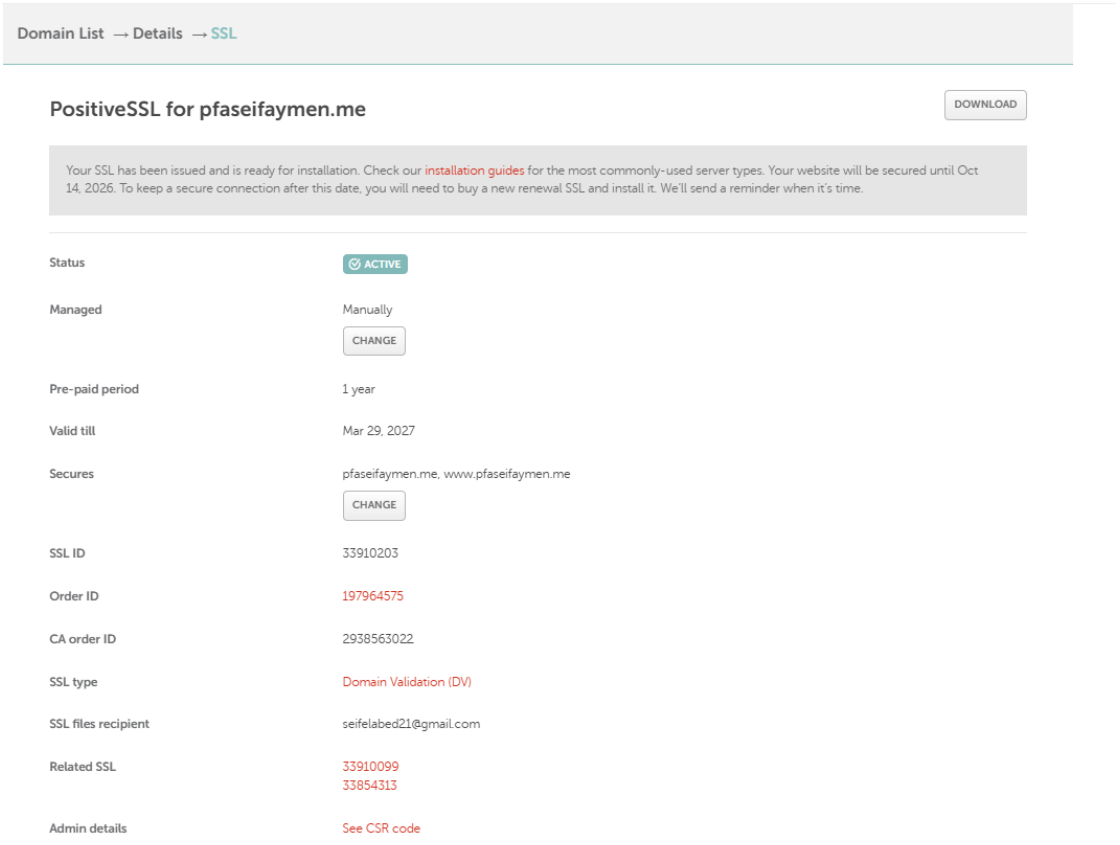


Figure A.15: *TLS - Certificat valide sur l'entree publique*

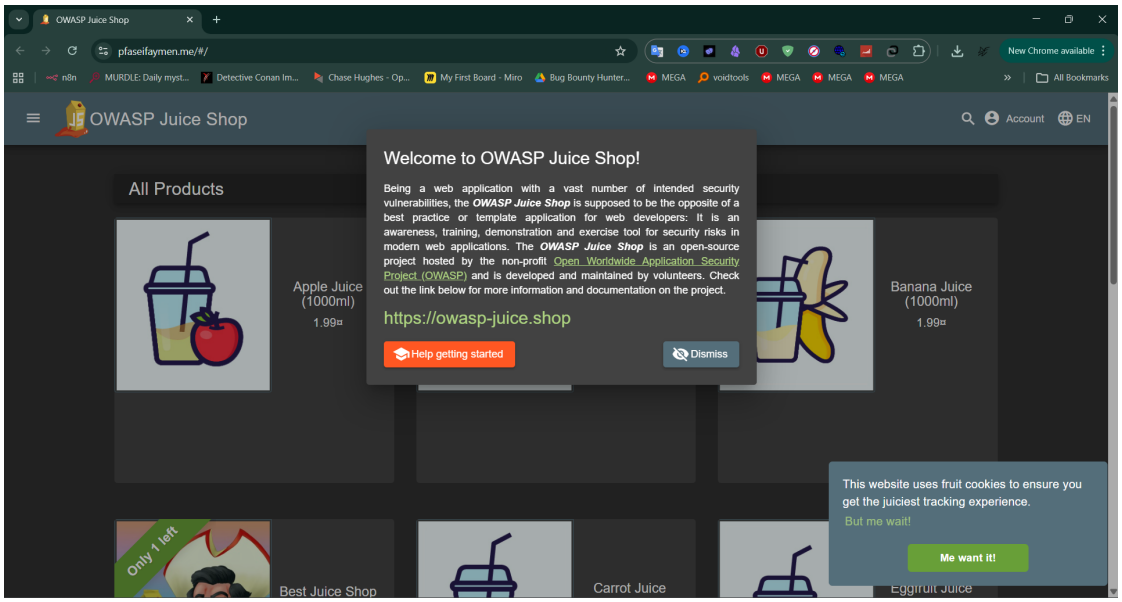


Figure A.16: *Domaine 1 - Securisation HTTPS validee*

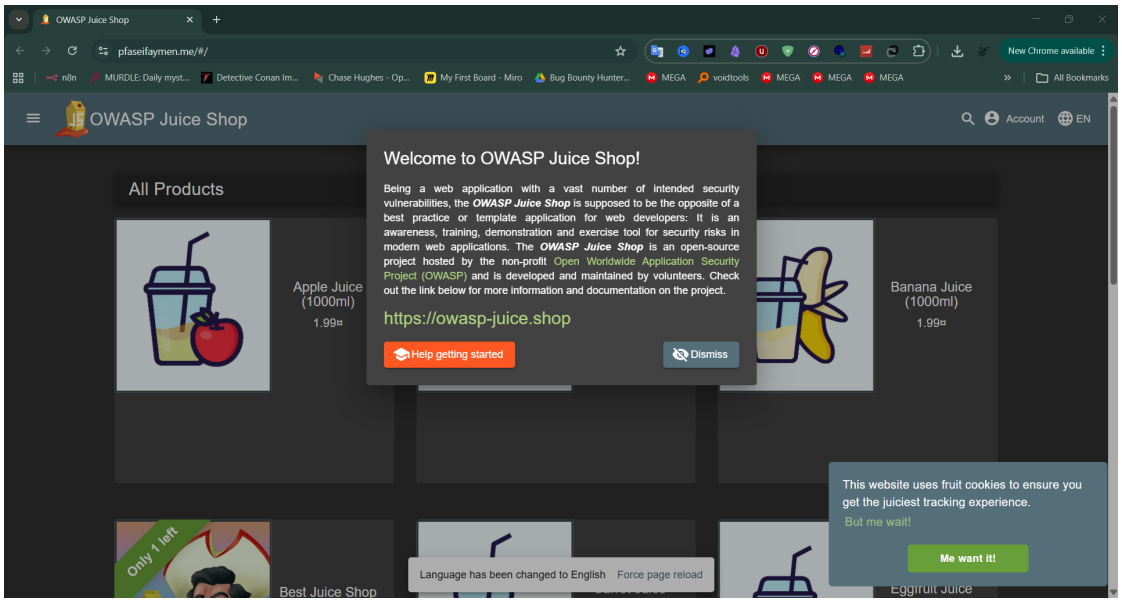


Figure A.17: *Domaine 2 - Securisation HTTPS validee*

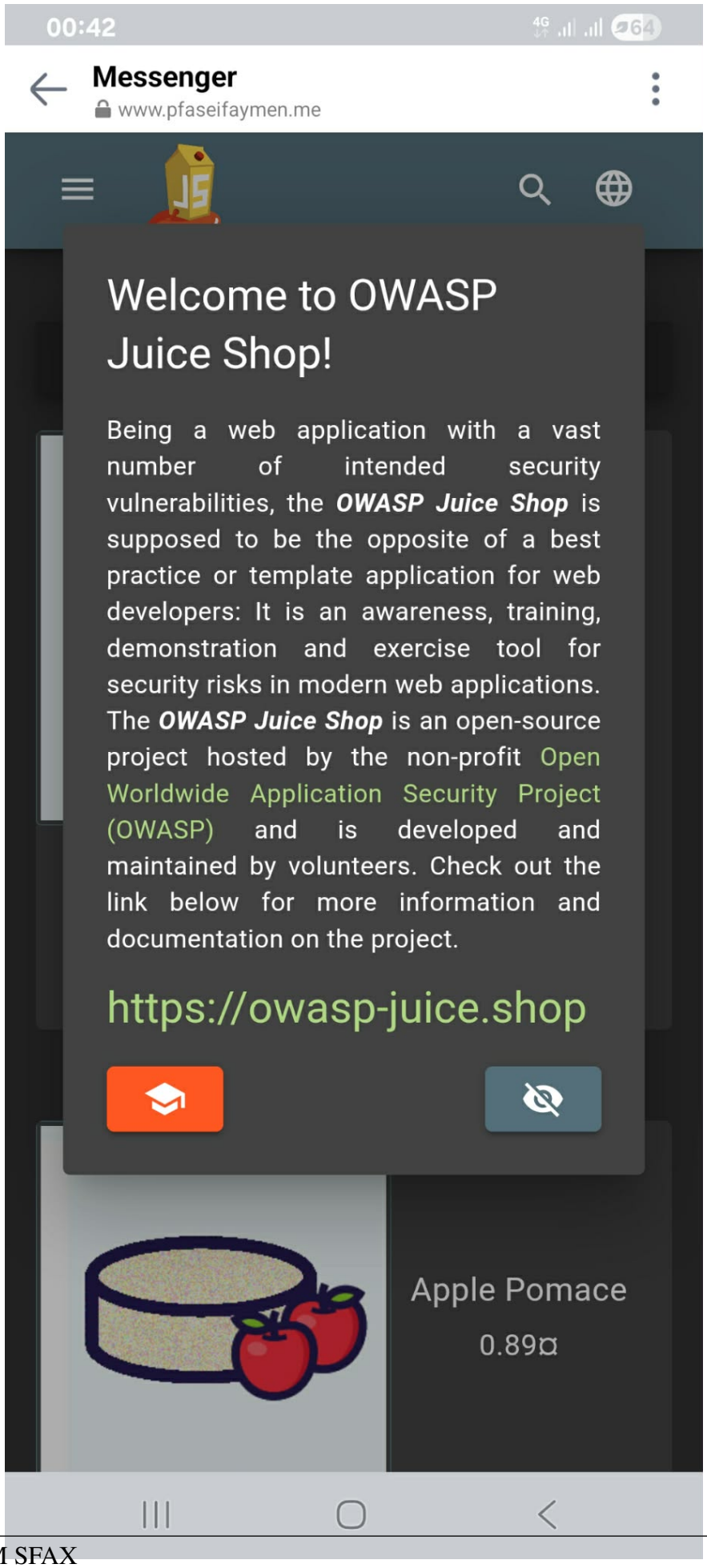


Figure A.18: *Mobile - Accessibilite securisee sur le domaine*

Securisation d'une Infrastructure Zero-Trust

Seifeddine El Abed
Aymen Gharbi

Résumé:
Ce projet presente la conception, la mise en oeuvre et la validation d'une infrastructure cloud securisee sur Azure selon une logique Zero-Trust et Security as Code. L'architecture adoptee combine Application Gateway WAF v2, Azure Firewall, NSG, VPN Point-to-Site et gouvernance TLS via Key Vault. Les ressources ont ete deployeees et maintenues avec Terraform afin d'assurer la reproductibilite et la traçabilite des changements. L'evaluation de sécurité a ete conduite par des campagnes d'attaques manuelles et automatisees (SQLi, XSS, fuzzing), suivies d'une analyse des journaux WAF et d'actions de remediation ciblees. Les resultats montrent une reduction significative de la surface d'attaque, une meilleure maitrise des flux egress, et une amelioration mesurable de la posture defensive globale.

Mots clés: Zero-Trust, Azure, WAF, Firewall, NSG, VPN P2S, Terraform, Security as Code, SQLi, XSS.

Abstract :
This project presents the design, implementation, and validation of a secure Azure cloud infrastructure based on a Zero-Trust and Security-as-Code approach. The adopted architecture combines Application Gateway WAF v2, Azure Firewall, NSG segmentation, Point-to-Site VPN, and TLS governance through Key Vault. Infrastructure provisioning and lifecycle management were implemented with Terraform to ensure reproducibility and traceability. Security effectiveness was assessed through manual and automated attack campaigns (SQLi, XSS, fuzzing), correlated with WAF logs and followed by targeted remediation actions. Results show a significant reduction of attack surface, stronger egress governance, and a measurable improvement of the overall defensive posture.

Key-words: Zero-Trust, Azure, WAF, Firewall, NSG, P2S VPN, Terraform, Security as Code, SQL Injection, XSS.

تلخيص:
يقدم هذا المشروع تصميم وتنفيذ والتحقق من صحة بنية تحتية سحابية آمنة على منصة أزور وفقاً لمنطق "الثقة المعدومة" والأمان كبرمجة. تعتمد البنية المعتمدة على دمج بوابة التطبيقات مع جدار حماية التطبيقات الويب، وجدار حماية أزور، ومجموعات أمان الشبكة، وشبكة خاصة افتراضية من نقطة إلى موقع، بالإضافة إلى إدارة أمان طبقة النقل عبر خزنة المفاتيح. تم نشر الموارد وإدارتها باستخدام تيرا فورم لضمان قابلية التكرار والتتابع. تم تقييم الأمان من خلال حملات هجوم يدوية وآلية (مثل حقن لغة الاستعلامات البنيوية، و البرمجة النصية عبر المواقع، والكشط العشوائي)، تلاها تحليل سجلات جدار حماية التطبيقات الويب وإجراءات معالجة مستهدفة. أظهرت النتائج تقليلاً كبيراً في سطح الهجوم، وتحكماً أفضل في التدفقات الصادرة، وتحسناً ملموساً في الوضع الدفاعي العام.

المفردات الدالة:
الثقة المعدومة، أزور، جدار حماية التطبيقات الويب، جدار حماية، مجموعات أمان الشبكة، شبكة خاصة افتراضية من نقطة إلى موقع، تيرا فورم، الأمان كبرمجة، حقن لغة الاستعلامات البنيوية، البرمجة النصية عبر المواقع